

ML2303 Project Report

Smart Customer Monitoring and Buffet Monitoring

Azar Rzayev, Faeza Shafika Sulianto



1. Introduction	3
2. Theoretical Framework and System Integration	4
2.1 Cyber-Physical Systems	4
2.2 Digital Maturity and Preconditions	7
2.2.1 System Context for Maturity Analysis	8
2.2.2 TOE Framework Application	8
2.2.3 Maturity Benchmarking with acatech Index	10
2.3 Data Analysis and Visualisation Tools	11
2.3.1 Smart Buffet Monitoring System	11
2.3.2 Smart Table & Queue Monitoring System	13
2.3.3 ILO3 Alignment and Reflection	13
2.4 Lean Principles and Operational Integration	13
2.5 Business Model Innovation and Servitization (ILO5) - shafa	14
2.6 Ethics, Privacy and Security	16
3. System Design and Implementation	18
3.1 Smart Table Management System	18
3.1.1 Hardware Configuration	18
3.1.2 Software Architecture	19
3.1.3 Data-Flow Design	19
3.1.4 Dashboard Interface	20
3.1.5 Design Choices	21
3.2 Queue Monitoring System	21
3.2.1 Hardware Configuration	22
3.2.2 Software Architecture	22
3.2.3 Data-Flow Design	23
3.2.4 Dashboard Interface	23
3.2.5 Design Choices	24
3.3 Smart Buffet Monitoring System	24
3.3.1 Hardware Configuration	24
3.3.2 Software Architecture	25
3.3.2.1 Tray-Classification and Serial-Forwarding Script	25
3.3.2.2 Node-RED Workflows	26
3.3.3 Dashboard Interface	27
3.4 System Integration	28
4. Discussion and Analysis	29
4.1 Operational Impact and Sustainability	29
4.2 Cross-Sector Transferability	31
5. Conclusion and Reflection	33
5.1 Key Outcomes and Impact	33
5.2.1 Smart Buffet Monitoring System	34
5.2.2 Smart Table and Queue Monitoring System	34
5.3 Conclusion	35
References	36
Appendices	38

1. Introduction

The food service and hospitality industry is pushed by regulatory demands to reduce and monitor food waste and also by customers to follow up with the world that is moving faster, granting the hospitality industry to be more responsive and efficient in giving services (Papargyropoulou et al., 2014; Jones et al., 2008). One clear challenge is managing customer flow, table turnover, and food availability without disrupting the dining experience. With unpredictable customer arrival rates and no digital tracking systems in place, restaurant operators often make reactive decisions based on visual inspection and personal judgment, leading to inefficiencies, long waiting times, unnecessary food waste, and poor resource allocation (Silvennoinen et al., 2015).

Digitalisation offers an opportunity to transform these operations by integrating real-time data, automation, and intelligent monitoring into restaurant workflows. Digital tools have already demonstrated success in manufacturing and logistics (Monostori, 2014), and similar benefits can be realized in hospitality by applying lean principles through Cyber-Physical Systems (CPS) and Industrial Internet of Things (IIoT). Examples include smart kitchens with connected appliances, digital reservation systems, and mobile ordering platforms (Panwar et al., 2019).

In response to these challenges, this project proposes a digital solution tailored specifically for buffet restaurant operations. The system integrates three interconnected components. First, a table management module detects the status of tables to display whether they are free, occupied, or require cleaning by using ultrasonic sensors and data logic processed via Node-RED. Second, a queue monitoring system utilizes a camera and vision-based pose estimation to track customers entering and leaving the waiting area, thereby calculating real-time wait times. Lastly, a buffet monitoring component aims to track food tray levels using image-based analysis, humidity, and temperature based sensors to alert staff for timely replenishment.

These modules aim to address the following operational problems:

- No visibility on customer arrival rate and table turnover.
- No optimized seating or reduced waiting times during peak hours.
- No digital data to analyze seating efficiency or correlate arrivals with food preparation.
- Decisions made based on guesswork rather than data.
- No feedback loop guiding food refills, leading to waste or shortages.
- Lack of hygiene alerts regarding unsafe food conditions.

The research goal for this report can be summarized into answering the research questions below:

1. How can digital system solutions be implemented in a restaurant setting to improve operational visibility and efficiency?
2. What are the key design considerations in selecting sensors and data flow systems for queue and table monitoring?

3. How can digital data from CPS modules be used to inform service strategies and reduce food waste in real-time?

The development and initial evaluation of this prototype involved multiple stakeholders and sources of feedback that helped shape the system design and potential use cases. Two team members led the technical implementation: Azar Rzayev, who focused on the buffet monitoring subsystem, and Faeza Shafika Sulianto, who was responsible for the table and queue monitoring subsystem. Both of us brought engineering and system design experience to the project and collaborated throughout the development phase to ensure the components could work together as an integrated digital restaurant solution.

Crucial practical insights were gathered through informal interviews conducted at *Taste by Nordrest*, a buffet restaurant located within the KTH campus. The team visited the restaurant during lunchtime hours and held short discussions with the restaurant manager and kitchen staff. These conversations provided direct input on operational pain points, such as difficulties in predicting tray depletion, food overproduction, and staff delays in identifying when tables were free or when a queue was building up. This feedback helped validate the real-world relevance of the monitoring systems and informed design decisions such as alert timing, interface simplicity, and the placement of sensors.

In addition, we benefited from business and innovation support through KTH Innovation. During a business coaching session, we received expert feedback on their early-stage business model. The Business Model Canvas was critically reviewed and refined, helping clarify the customer segment, key value propositions, and possible revenue streams. The coach emphasized the importance of demonstrating scalability and practical impact, particularly in large-scale food service environments such as hotels, universities, and hospitals. The session also encouraged us to position our system not merely as a technical prototype but as a potential data-driven service platform for reducing food waste and improving operational efficiency in the hospitality sector. Together, these technical and stakeholder inputs contributed to a solution that is both grounded in real-world restaurant needs and aligned with emerging business opportunities in sustainable and digital food service operations.

This digital system is developed using a combination of Arduino-based ultrasonic sensors, Raspberry Pi edge computing, camera-based tracking, Node-RED data flow design, and Google Sheets/Firebase for cloud logging. The solution is intended to demonstrate how digital technologies can enhance lean operations and support sustainable restaurant practices.

This project aims to support digital transformation in restaurants by integrating real-time monitoring, automation, and data analytics. The solution promotes lean operations, reduces waste, and enables new service-oriented business strategies. It also addresses ethical and data security considerations when deploying digital tools in customer-facing environments.

In the chapters that follow, we will detail the theoretical foundations (e.g., CPS, Industry 4.0), describe the system architecture and implementation, analyze its effectiveness, and reflect on broader implications for digitalisation in hospitality.

2. Theoretical Framework and System Integration

2.1 Cyber-Physical Systems

A Cyber-Physical System (CPS) is a system in which physical processes and digital technologies are closely integrated and interact with each other in real-time. These systems are monitored and controlled through embedded computing and communication technologies. CPS is considered one of the core enablers of Industry 4.0, providing a technological foundation for intelligent automation and smart systems across various sectors including manufacturing, logistics, and services (Lee et al., 2015).

CPS typically consists of six main components that work together to bridge the gap between the physical and digital world:

- **Sensors and Actuators:** These serve as interfaces between the real world and the digital system. Sensors gather real-time data from the environment, while actuators execute actions based on decisions made by the system.
- **Embedded Systems and Edge Devices:** Devices such as Raspberry Pi, microcontrollers, or PLCs locally process data collected by sensors, allowing immediate responses without relying on cloud connectivity.
- **Communication Networks:** Networks like Wi-Fi, MQTT, and 5G enable reliable and real-time data transmission between physical components and the digital control systems.
- **Data Processing and Analytics:** Collected data is analyzed using traditional algorithms or AI methods to extract useful insights and support intelligent decision-making.
- **Control Systems:** These manage system behavior through feedback loops, allowing the CPS to adapt automatically to changes in the environment.
- **Cybersecurity Frameworks:** Security mechanisms ensure that data and operations remain confidential, resilient, and tamper-proof.

By combining these elements, CPS enables real-time monitoring, autonomous decision-making, and intelligent control of physical systems. Emerging technologies like digital twins further extend CPS capabilities by allowing virtual representations of physical assets to be used for simulation, optimization, and predictive maintenance. This integration increases efficiency, flexibility, and transparency in operations (Monostori et al., 2016).

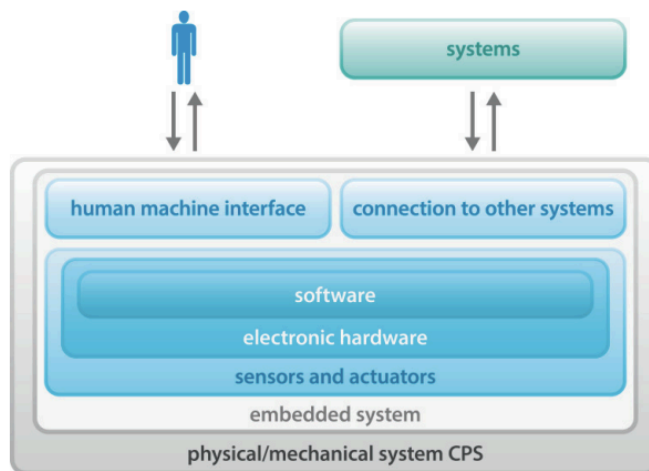


Figure 1. Interaction between humans and machines in cyber physical systems (Monostori, L. (2014))

CPS can also serve as an indicator of an organization's readiness for digital transformation. Tools like the Acatech Industry 4.0 Maturity Index (Schuh et al., 2017) can be used to evaluate how prepared a company is for adopting advanced digital technologies.

Beyond operational efficiency, Cyber-Physical Systems (CPS) provide a range of strategic benefits that enhance overall system performance and business value. One key advantage is the automation of repetitive tasks, which helps reduce labor costs while improving consistency and accuracy in operations. Additionally, CPS can significantly enhance customer experience by enabling personalized services and minimizing wait times through real-time responsiveness. Moreover, CPS supports sustainability goals by reducing waste, optimizing resource utilization, and promoting more efficient energy and material consumption throughout the system.

The smart restaurant system developed in this project applies the core principles of Cyber-Physical Systems (CPS) by integrating physical hardware components, embedded computing logic, and a cloud-connected user interface. This integration enables real-time monitoring and intelligent decision-making for table management in the restaurant environment.

At the physical layer, ultrasonic sensors (HC-SR04) are installed above each dining table to detect customer presence by continuously capturing distance measurements, which reflect real-time table occupancy. These sensor readings are sent to an Arduino microcontroller acting as the data acquisition unit, converting physical conditions into digital signals for further processing. Similarly, the queue monitoring system employs a USB webcam positioned to track customer flow at the ordering counter, using computer vision techniques to identify queue length and wait times in real time. The smart buffet monitoring system uses an overhead camera combined with environmental sensors (temperature and humidity) connected via Arduino to assess tray fill levels and ambient conditions. Together, these sensing components form the physical layer inputs that enable the broader CPS to monitor, analyze, and respond dynamically to changing conditions within the restaurant environment.

The cyber layer consists of a Raspberry Pi, which receives data from the Arduino via serial communication. On the Raspberry Pi, the Node-RED programming platform is used to interpret the incoming data and apply decision-making logic. This logic determines the status of each table whether it is Free, Occupied, or Needs Cleaning. The system also records the timestamp for each status change and calculates how long a table has remained in a particular state. In addition, it logs this information into Google Sheets, which functions as a cloud-based database for long-term tracking and analysis. This layer is responsible for processing sensor data, executing decision logic, and enabling intelligent responses based on real-time conditions.

The user interface layer is implemented through a web-based Node-RED Dashboard. This dashboard displays the current status of each table, how long the table has remained in its current state, and a historical log of status changes. All relevant data is simultaneously pushed to Google Sheets in the cloud, which allows for remote access, trend analysis, and forecasting. This interface supports restaurant staff by providing actionable insights, such as identifying which tables require cleaning or monitoring occupancy trends to optimize customer service. Through this visual representation, the system enhances situational awareness and facilitates informed human decision-making.

Together, these layers form a closed feedback loop, characteristic of a fully functional CPS. The cycle begins with sensors detecting physical data, which is then interpreted by the embedded software. The processed data is visualized on the dashboard, allowing staff to respond with appropriate actions such as cleaning a table or preparing it for the next guest. These actions result in changes to the physical environment, which are again captured by the sensors, thereby completing the loop. This continuous cycle enables the system to adapt dynamically to real-world changes and maintain efficient operations in the restaurant.

The implementation of this smart system demonstrates how CPS technologies can enhance service efficiency, streamline operations, and support data-driven decision-making in a service-oriented environment such as a restaurant.

2.2 Digital Maturity and Preconditions

Digital maturity refers to an organisation's ability to adopt and integrate digital technologies in a way that improves operations, enhances responsiveness, and supports long-term strategic goals. In the context of smart services, this maturity involves not only technology readiness but also organisational capability, data literacy, and user experience design (Dumitrescu et al., 2023). Within the hospitality sector, digitalisation has largely focused on customer-facing tools, while operational tasks such as table coordination, hygiene monitoring, and buffet refilling often remain disconnected from real-time data sources (Buhalis & Leung, 2018; Nikopoulou et al., 2023).

To evaluate the digital maturity and contextual preconditions for innovation in buffet-style dining, we applied two conceptual frameworks: the Technology–Organization–Environment (TOE) model and the acatech Maturity Index for Smart Services. The TOE framework, first introduced by Tornatzky and Fleischer (1990), offers a structured way to examine how innovation is influenced by technological infrastructure, organisational capabilities, and external environmental conditions. Although originally developed for industrial and IT settings, its emphasis on practical adoption factors makes it relevant

for hospitality contexts, where digital tools often face challenges such as limited budgets, disruption to existing workflows, or lack of digital readiness among staff.

The acatech maturity model (acatech, 2023) complements this perspective by describing digital transformation as a staged process. It ranges from basic computerisation and connectivity to full adaptability, where systems are able to respond autonomously based on integrated data. While the model was developed for advanced manufacturing and service industries, we used it conceptually to assess the current state of buffet-style operations and to understand where our proposed system might fit within a broader digital transformation pathway

2.2.1 System Context for Maturity Analysis

Our project addresses operational inefficiencies in dining environments through two connected systems. These were developed based on early-stage dialogues with staff from Taste by Nordrest, an institutional dining provider at KTH. During our informal discussions and observations, we learned that most coordination was done verbally or by simply checking visually, staff would walk around to see if trays were empty, if tables were ready, or if a queue was forming. While this approach works in practice, it creates inefficiencies, especially during peak service times. To address this, The Smart Buffet Monitoring System has been developed which includes two core elements. First, a vision-based module detects the food level of trays using a machine learning model trained on more than 800 labelled images. This model classifies each tray state as Full, Medium, Low, or Empty. Second, temperature and humidity sensors are placed near trays to provide ongoing hygiene-related monitoring. These two data streams are visualised on a live dashboard that supports timely refill and sanitation decisions. While the vision system performs effectively at the prototype stage, its performance remains dependent on lighting conditions, tray positioning, and food type variability. The current model would benefit from expanded datasets and further calibration before being suitable for integration into live dining environments. Nonetheless, it demonstrates the potential of low-cost computer vision for food service monitoring.

The second solution is The Smart Table Management System combines ultrasonic sensors for detecting table occupancy and cleanliness with a vision-based queue tracking system. The queue tracking module uses pose estimation via MediaPipe to monitor the number and flow of individuals in the buffet service zone. To further enhance system utility, we implemented a basic method of distinguishing service staff from customers by analysing differences in shirt color, which tends to vary systematically between uniforms and visitor clothing. While this approach is functional at the prototype level, it has limitations and would require further model training and validation for reliable deployment in dynamic, real-world environments. Both the occupancy and crowding data streams are visualised through a shared dashboard interface that supports real-time decision-making by front-of-house staff.

2.2.2 TOE Framework Application

The TOE framework was used to identify the technological, organisational, and environmental factors that shape adoption potential in buffet-style dining. Table 1 summarises these elements based on our experience and stakeholder input.

Table 1. Key TOE factors in buffet-style dining environments.

TOE Dimension	Contextual Characteristics
Technology	Use of machine learning for tray monitoring, environmental sensors, ultrasonic sensing for occupancy, and pose estimation for crowd tracking
Organisation	Low tolerance for complex systems, strong reliance on verbal coordination, limited training time, and resistance to tools perceived as slowing down workflows
Environment	Stringent hygiene requirements, cost constraints, short service windows, and a general conservatism toward adopting new technologies

This mapping helped us reflect on both the design strengths and maturity limitations of our prototype. The buffet monitoring system combines multiple sensor types (vision and environmental) to support a more responsive kitchen process. The table and queue monitoring system similarly combines ultrasonic and vision data to assist staff. However, in terms of acatech stages, both systems remain at an early stage of maturity, primarily between the “connectivity” and “visibility” levels. Data is collected, interpreted, and visualised in dashboards, but higher levels like predictive analytics and cross-system adaptability are not yet implemented.

To contextualise the potential for further development, we reviewed a hospitality data architecture proposed by Buhalis and Leung (2018), which outlines how hospitality ecosystems can evolve through the integration of internal, partner, and environmental data layers. This framework helped us reflect on the future scalability of our system, particularly in institutional catering environments or hotel chains where data from multiple service points could be unified.

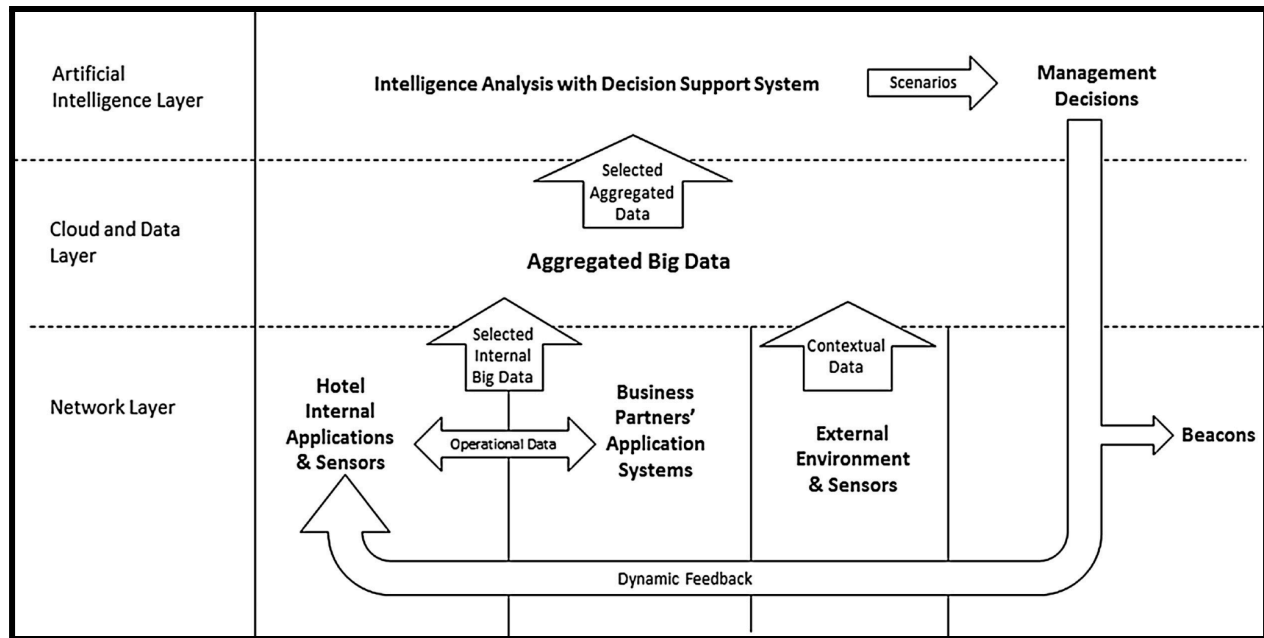


Figure 2. Hospitality Smart Service Architecture. Adapted from Buhalis and Leung (2018)

While our prototype operates as a standalone system, it shares conceptual elements with this architecture. For example, both subsystems generate operational data that could be combined with external indicators (e.g., guest traffic, scheduled events) to support higher-level planning. This reinforces the idea that even limited, low-cost prototypes can serve as foundational blocks toward more integrated, digitally mature service ecosystems.

2.2.3 Maturity Benchmarking with acatech Index

To further evaluate the digitalisation potential of our proposed systems, we applied the acatech Maturity Index for Smart Services (acatech, 2023). This framework defines six levels of maturity, including Digital Starter, Smart Product Manufacturer, and Smart Solution Integrator. Based on our observations and stakeholder insights, current practices in many institutional and commercial buffet operations, including those at Taste by Nordrest correspond to the Digital Starter level. These environments typically lack integrated sensor systems, automated alerts, or unified interfaces.

In contrast, the prototype systems developed in this project introduce functionality consistent with more advanced maturity levels. These include automated data generation from sensors, minimal need for manual input, and real-time visualisation for decision support. Table 2 summarises this comparison across several operational dimensions.

Table 2. Functional maturity positioning based on current practices and proposed system capabilities

Function Area	Current Practice	Target Capability	Mapped Maturity Stage
Table turnover coordination	Manual inspection and verbal updates	Dashboard showing table occupancy and cleanliness using sensors	Digital Starter
Queue awareness	Staff observation and guesswork	Pose-based vision tracking and basic staff/customer distinction	Smart Product Manufacturer
Buffet refill timing	Tray status judged visually	Vision-based tray classification and refill alerts	Smart Product Manufacturer
Hygiene monitoring	Periodic checks by staff	Temperature and humidity sensors with dashboard output	Smart Product Manufacturer
Cross-system integration	Fragmented information sharing	Unified dashboard connecting tray, table, and customer flow data	Smart Solution Integrator

2.3 Data Analysis and Visualisation Tools

This project made extensive use of data analysis pipelines and stakeholder-specific visual interfaces to support both immediate operational decisions and long-term strategic planning. By combining real-time sensing, flow-based programming, and modular dashboards, we aimed to demonstrate how digital tools can enhance responsiveness and insight across different roles in buffet-style dining environments. These practices align with ILO3, which emphasizes the application of data analytics, AI, and stakeholder-oriented visualisation tools in production and service contexts (Nikopoulou et al., 2023; Buhalis & Leung, 2018).

2.3.1 Smart Buffet Monitoring System

The buffet subsystem integrated two core data sources: a vision-based tray level detector and environmental monitoring sensors. A webcam was used alongside OpenCV to isolate the tray region, after which a TensorFlow image classification model was trained on approximately 800 labeled images and it was applied to classify food level as Full, Medium, Low, or Empty. The shift from using load sensors (e.g., HX711) to this vision-based solution was due to inconsistencies in sensor readings during early testing.

In parallel, DHT22 sensors monitored temperature and humidity conditions near the trays. All data was streamed to Node-RED, where it was processed using function and switch nodes to extract values, filter noise, and trigger real-time alerts. For example, if the tray status was “Low” and the temperature exceeded 30°C, the system generated a high-priority hygiene alert.

This information was visualised using a custom-built HTML/CSS/JavaScript dashboard, designed for kitchen staff. The interface displayed tray status cards with color-coded indicators, real-time temperature and humidity graphs, and timestamped updates.

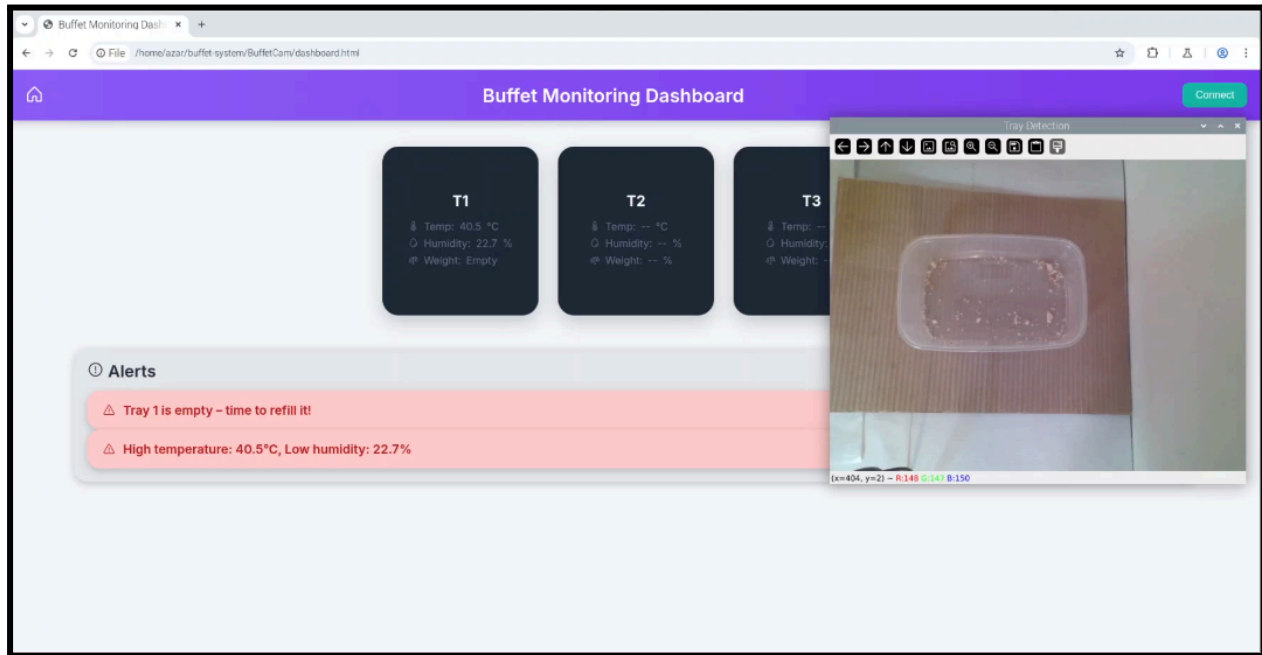


Figure 3. Buffet Monitoring Dashboard for Kitchen Staff. Source: Author’s own work.

Manual override buttons were included to allow staff to confirm refill actions, promoting system adaptability. All data was logged to a Firebase Realtime Database, structured by tray ID, status, environmental readings, override flags, and timestamps.

To support longer-term insights, the collected data was periodically exported to Power BI, where an analytics dashboard was created for managerial use. This dashboard included KPIs such as:

- Average refill time per tray
- Peak refill hours by weekday
- Temperature anomalies
- Refill frequency heatmaps

This dual-dashboard structure aligns with the recommendation by Nikopoulou et al. (2023), who emphasize the need for role-specific visualisation tools that separate operational monitoring from strategic decision-making.

2.3.2 Smart Table & Queue Monitoring System

The table management subsystem used ultrasonic sensors (via Arduino) to detect whether tables were free, occupied, or needed cleaning. In parallel, a vision-based queue tracking system was implemented using MediaPipe pose detection and shirt color filtering, enabling identification of guest entries and exits near the waiting area.

All data was processed through Node-RED using serial and JSON inputs, logic nodes, and dashboard elements. A web-based dashboard displayed table statuses with live color updates and manual confirmation buttons for cleaning events. Queue events were logged with timestamps and aggregated into occupancy counters.

The data from both systems was logged in Google Sheets, including table ID, status history, and queue metrics. The structured nature of this dataset creates an opportunity to develop a dedicated analytics dashboard in Power BI, enabling managers to track operational performance over time. Potential KPI metrics could include average table turnaround time, cleaning response delays, and queue–table availability mismatches, supporting data-driven staff scheduling and service optimization.

2.3.3 ILO3 Alignment and Reflection

Through the use of advanced data tools such as TensorFlow, OpenCV, Node-RED, Firebase, Power BI, and Google Sheets, the project demonstrated how real-time and historical data can be transformed into role-specific insights. By designing both operational dashboards for frontline staff and analytic models for managerial decision-makers, the system reflects the ILO3 goal of integrating data analysis, machine learning, and visualisation into production-like workflows in the service domain.

2.4 Lean Principles and Operational Integration

The Smart Restaurant Digital System applies lean principles to improve operational efficiency, reduce waste, and support sustainable practices. Lean thinking focuses on identifying and eliminating activities that do not add value, such as excess waiting, unnecessary motion, and overproduction (Womack and Jones, 1996).

Food waste is a significant concern in the hospitality industry. In 2021, total food waste in the European Union reached 58.4 million tonnes, with restaurants and food services responsible for about 9 percent of this total, or roughly 5.26 million tonnes (Eurostat, 2023). This level of waste highlights the need for sector-wide interventions that address overproduction and spoilage.

Our buffet monitoring subsystem targets food waste by using a trained TensorFlow model to classify tray levels (Full, Medium, Low, Empty). Whenever a tray falls to Low or Empty, a real-time alert appears on the kitchen dashboard, prompting staff to refill only those trays. In traditional buffet operations, refills often follow fixed schedules or manual checks, leading to excess food preparation and the risk of running out at peak times. By basing refills on actual consumption data, this approach aligns with lean's pull-based principle.

A pilot program by the Sustainable Hospitality Alliance in 2019 by using the Leanpath digital food-waste tracking solution which asked hotel kitchens across ten European properties to weigh and record discarded food using tablets and software. Over four months, this continuous logging and staff training led to a 64 per cent drop in measured kitchen food waste, as teams identified which dishes or serving practices generated the most waste. Another study by Bui and Nair (2022) installed a simple scale-and-display system in four self-service buffet kitchens. Each time diners discarded leftovers, staff recorded the weight on an in-kitchen monitor. After one year, those kitchens reduced their buffet-related waste by around 1,800 kilograms per location annually. Although our system has not been tested at scale, these pilots provide a basis for a conservative guestimate of a 25 percent reduction in buffet-related food waste for institutional canteens and large-scale restaurants.

The Smart Table and Queue Monitoring subsystem also incorporates lean concepts by reducing waiting and motion waste. Ultrasonic sensors detect table occupancy and cleanliness, while a separate camera based module monitors customer flow. Dashboards then display each table's status: Occupied, Free, or Needs Cleaning, and current queue length, allowing staff to respond immediately. In many dining settings, staff conduct periodic rounds, which can result in unnecessary walking and slower table turnover. Instead, real time alerts direct them straight to tables that require attention. Research indicates that real time dashboards improve lean operations by increasing visibility and reducing decision delays (Rossini et al., 2019).

Both subsystems rely on visual management through their respective web dashboards. The buffet dashboard uses color coding to show tray status, such as a red outline for Low food, while the table management dashboard highlights tables needing cleaning in yellow. These visual controls remove ambiguity and standardize staff responses, supporting continuous flow and ensuring that tasks occur only when actual demand requires action rather than on a fixed schedule.

In summary, the Smart Restaurant Digital System demonstrates how lean principles can be operationalised through digital tools. By integrating vision based tray detection, environmental sensors, ultrasonic sensing, and real time dashboards, the system delivers just in time service, minimises food waste, reduces unnecessary waiting and motion, and optimises workflow. These enhancements align directly with ILO4 on operational integration and efficiency improvement.

2.5 Business Model Innovation and Servitization (ILO5) - shafa

Business model innovation is a critical process for companies navigating digital transformation. One widely used framework for describing and developing business models is the Business Model Canvas (BMC). The BMC consists of nine interconnected building blocks that describe how a company creates, delivers, and captures value. These blocks include Customer Segments, Value Propositions, Channels, Customer Relationships, Revenue Streams, Key Resources, Key Activities, Key Partnerships, and Cost Structure. This structured visual tool allows organizations to conceptualize and communicate the core components of their business logic, especially useful when integrating digital technologies or exploring servitization models.

The adoption of digital technologies in the restaurant and hospitality sector has opened new pathways for business model innovation, especially through the concept of servitisation. Rather than solely

selling physical products or systems, companies are now offering integrated services that deliver ongoing value through data insights, predictive alerts, and cloud-based monitoring. This shift is particularly relevant in smart environments where physical and digital layers are closely integrated.

In the case of our smart restaurant system, the core business model has transitioned from offering hardware-based monitoring solutions to delivering a subscription-based digital service. The latest Business Model Canvas (BMC) illustrates this transformation. The value proposition is centered on optimizing table turnover, providing real-time operational insights, ensuring hygiene through temperature alerts, and assisting staff during peak periods with automated signals. These features are made possible by combining sensors, edge computing, cloud dashboards, and feedback loops.

The servitization strategy is implemented through a Software-as-a-Service (SaaS) approach, where restaurant clients subscribe to continuous access to system dashboards, real-time analytics, and automated alerts. This model creates recurring revenue and fosters a long-term relationship between provider and client. Additionally, premium support packages and custom integrations are available to tailor the system to individual restaurant needs.

Recent research supports this trend. According to Lerch and Gotsch (2020), digital servitization enables manufacturers and service providers to move up the value chain by embedding digital functionalities that provide continuous value. Similarly, Kohtamäki et al. (2020) argue that the integration of digital platforms and analytics into traditional product offerings leads to stronger customer relationships and greater differentiation.

In our system, customer segments include buffet-style restaurants, hotels and resorts, institutional dining facilities, mid-sized restaurant chains, and food courts—reflecting a broader range of service-oriented food operations. The system’s flexibility and scalability make it suitable for diverse environments where service speed, hygiene, and operational efficiency are mission-critical.

Channels to reach these customers include direct outreach, social media marketing, and partnerships with POS and kitchen equipment vendors. Feedback mechanisms such as analytics dashboards and self-service help centers maintain customer relationships and inform continuous improvement.

By aligning digital capabilities with service logic, our smart restaurant system demonstrates how servitization can enable not only efficiency and sustainability but also business growth in the hospitality sector. *See appendix A*

The full integration of queue monitoring, table turnover tracking, and food safety monitoring creates a holistic service platform that delivers significant value across the entire operation. In a manufacturing or production context, this would be similar to integrating worker flow tracking, machine utilization monitoring, and inventory condition management. For example, instead of monitoring diners in a restaurant, sensors and computer vision could be deployed to observe worker distribution in an assembly line, identifying bottlenecks or idle zones. Similarly, table turnover logic can be repurposed to monitor workstation availability, flagging areas needing cleaning, restocking, or maintenance before the next task cycle.

This type of data-driven coordination enables smarter scheduling and equipment allocation, thereby enhancing throughput and reducing waste. This modular and interoperable architecture allows for scalable deployment across production units, warehouses, or even cross-site logistics, while maintaining alignment with lean production.

From a production perspective, this solution supports lean operations by applying key principles such as waste reduction, flow optimization, and responsiveness to customer demand. By automating station tracking and workforce allocation, the system helps minimize downtime and ensures optimal use of resources aligning with lean manufacturing's goal of eliminating non-value-added activities. The real-time data provided by these systems allows managers to adapt workflows dynamically and implement continuous improvement cycles based on factual insights. This integration of digital technologies with lean thinking reflects emerging trends in smart production environments where cyber-physical systems support agility, transparency, and real-time adaptability (Kamble, Gunasekaran, & Gawankar, 2023).

2.6 Ethics, Privacy and Security

Any monitoring system in a restaurant setting must balance improved efficiency with respect for people's privacy. Our prototype uses cameras and sensors to streamline buffet and table operations, but before rolling it out commercially we need clear plans for transparency, privacy protection, and legal compliance.

In a commercial buffet, customers and staff should see a simple notice explaining that cameras mounted above the trays only detect food levels such as Full, Medium, Low, or Empty and no images of people are stored. Putting up a sign near the buffet line will make it clear that the system only sends tray status data to the dashboard. For queue monitoring, a notice could say that the camera only looks at shirt color on the mid torso to count customers, never recording any faces or full body images. By being upfront about what the camera sees, we follow Privacy by Design principles (Cavoukian, 2010) and ensure everyone knows exactly what information is collected.

To protect individuals, the buffet camera is angled straight down over the tray area so it never captures customer faces. We store only "tray ID plus status," and no frames or still images are kept. In the queue area, the system uses pose detection but only saves a color code for each shirt and assigns an anonymous ID, nothing that could identify a person is recorded. These choices follow data minimization best practices (Spiekermann and Cranor, 2009). By avoiding any raw image storage, we greatly reduce the chance of identifying someone.

Once the data is collected, it should not linger unnecessarily. Under GDPR rules (EU 2016/679), personal data must be deleted as soon as it is no longer needed. In a commercial setting, raw logs of tray levels and queue counts can be kept only for about 24 hours for debugging or immediate analysis and then purged automatically. Aggregate numbers, such as daily totals of tray refills or hourly queue lengths, can be stored longer for reports and trend analysis but should not include any information that ties back to individual staff or customers. If temporary logs are used for calibration, they must be encrypted and only accessible to a small authorized team and set to overwrite after a brief period.

In the queuing monitoring system, similar privacy protections are applied to ensure that individuals are not identified or tracked inappropriately. The camera used for queue analysis is positioned to focus on the mid-torso region only, where it captures shirt color and pose landmarks, avoiding facial features or any biometric data. The system assigns each detected individual a random anonymous ID (e.g., "Person_001") and only logs high-level metrics such as time of entry, time of exit, and estimated wait duration. No video or still images are stored, and no attempt is made to track individuals beyond a single session. These design choices adhere to data minimization and purpose limitation principles under GDPR. Moreover, the system provides aggregate metrics (e.g., average wait time per hour) without retaining any personal or sensitive information. If temporary logs are used for debugging or model improvement, they are encrypted, locally stored, and deleted within 24 hours. Public signage should clarify that the camera system is used solely for crowd estimation, with no face recognition, recording, or identification involved further reinforcing transparency and compliance with Privacy by Design standards.

When it comes to design, the principle of Value Sensitive Design (van den Hoven et al., 2015) reminds us to involve stakeholders from the start. In a commercial rollout, we would meet with kitchen managers and floor staff to explain exactly how each sensor works, what it sees and what data it sends to the dashboard. Staff consent could be formalized in a short form stating that monitoring of service stations is only for operational purposes and not to track individual performance. Customers who do not want to be counted in queue data can avoid standing directly in front of the camera's view. All data must be stored securely. In a Firebase or similar backend, access controls are needed so that only designated staff (for example kitchen supervisors or floor managers) can see real time or historical data. Encryption in transit and at rest prevents unauthorized access if the database is breached (Li and Yang, 2017). Audit logs should track who viewed or downloaded data, helping demonstrate compliance with GDPR Articles 30 and 32 on accountability and security.

Because GDPR is the governing law (EU 2016/679), we must treat even indirect identifiers such as time stamped queue counts in a small café as personal data. Article 25 requires privacy by design from the very beginning. By limiting data collection to anonymous tray statuses or color coded counts, and by defining clear deletion rules, we follow GDPR's principles of lawfulness, fairness, transparency, and data minimization.

By taking these steps and making data collection visible, storing only minimal information, purging raw logs quickly, involving stakeholders early, and enforcing strict access controls, a commercial version of our system can operate ethically and legally while still delivering the efficiency benefits that justify its use.

3. System Design and Implementation

3.1 Smart Table Management System

This subsection presents the development and deployment of a smart table management system tailored for restaurant environments, aimed at improving operational transparency, enhancing staff response times, and supporting data-driven decision-making. The system provides real-time visibility of table occupancy and cleanliness status through a sensor-based detection mechanism integrated with a digital dashboard. It is structured across six key components: hardware configuration, software architecture, data-flow design, dashboard interface, rationale for design choices, and system limitations.

3.1.1 Hardware Configuration

Each table in the restaurant was fitted with an HC-SR04 ultrasonic distance sensor, positioned discreetly under the table and angled to monitor customer presence. The sensor's Trigger and Echo pins were connected to an Arduino Uno (D8 and D9, respectively), with the 5V and GND pins used for power supply. The Arduino continuously measured the distance to the nearest object (e.g., a seated customer's torso).

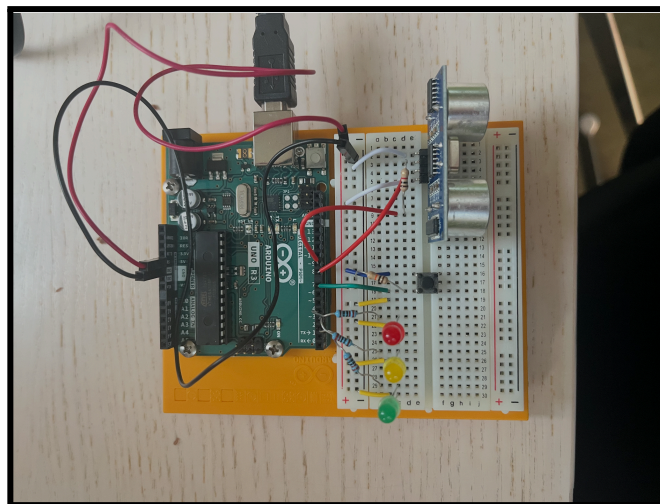


Figure 4. Arduino Ultrasonic Set. Source: Author's own work.

The detection thresholds were calibrated between 20 cm and 70 cm. Based on the sensor readings, the Arduino mapped the environment into one of three states:

- Free: No object detected within the threshold range.
- Occupied: Object detected within the threshold range.
- Needs Cleaning: A recent transition from "Occupied" to "Free," with a delay added before resetting to "Free" to indicate the table needs cleaning.

Initial calibration and testing were performed using the Arduino IDE’s serial monitor. After confirming the sensor behavior and logic, serial communication via USB was established between the Arduino and a Raspberry Pi for full integration with the data processing workflow.

3.1.2 Software Architecture

The software was designed using Node-RED, hosted on the Raspberry Pi, acting as the central control hub. The Node-RED flow included the following functional nodes:

- Serial Node: Receives distance values streamed from the Arduino.
- Parser Node: Cleans and extracts numeric data from incoming serial messages.
- Function Node: Contains JavaScript logic for state transitions. It uses real-time distance values, internal memory for tracking status changes, and a timer-based delay mechanism to identify the “Needs Cleaning” state.
- Switch Node: Detects changes in state to trigger downstream processes.
- Dashboard Output Nodes: Display current status using ui_text and ui_card elements.
- Google Sheets Node: Logs data such as table ID, new state, timestamp, and duration since the last state into a cloud-based spreadsheet for historical tracking and further analysis.

Each table's flow could be duplicated and modified to scale up the system. This modular structure facilitated debugging and allowed incremental deployment.

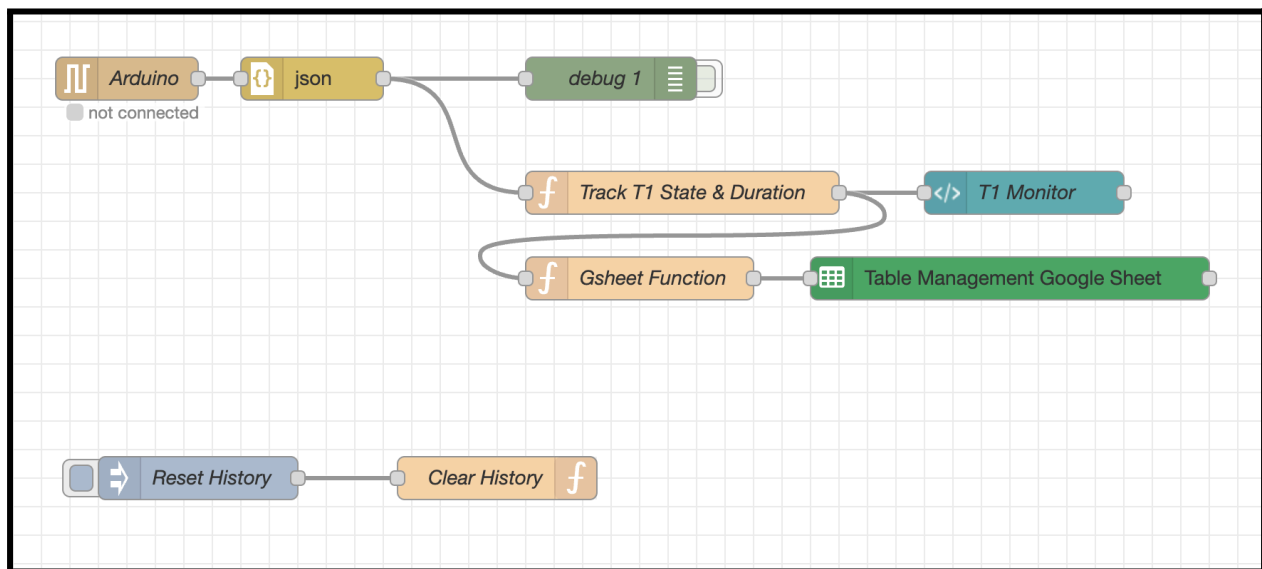


Figure 5. Table Management Node Red Flow. Source: Author’s own work.

3.1.3 Data-Flow Design

The real-time data flow is organized as follows: **Arduino → Serial (USB) → Raspberry Pi (Node-RED) → State Mapping → Dashboard Display + Google Sheets Logging**

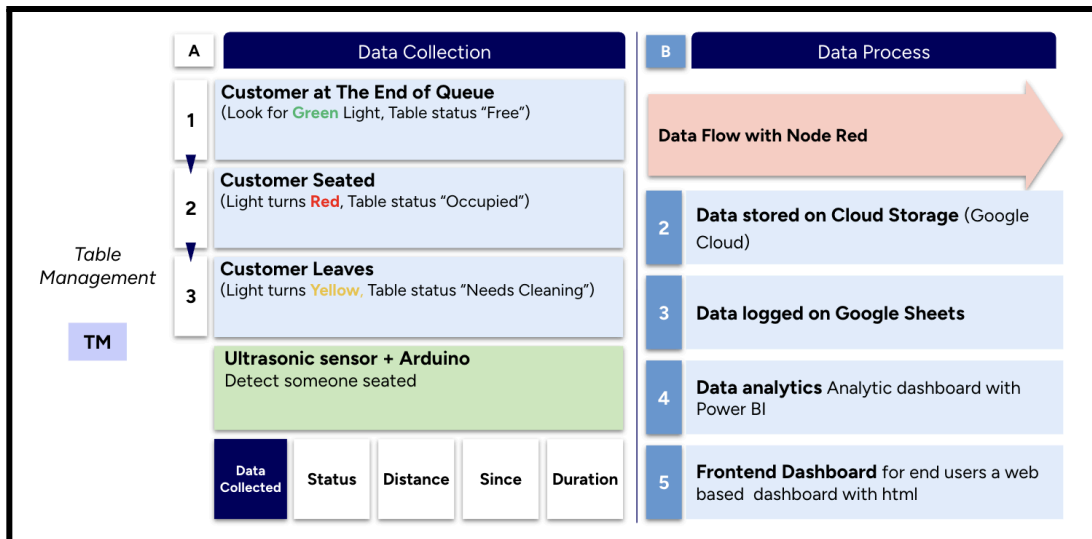


Figure 6. Table Management Data Flow. Source: Author's own work.

Every distance value is evaluated and mapped into a table status. When a state change is detected (e.g., from "Occupied" to "Needs Cleaning"), a new row is automatically written to the linked Google Sheet with the following fields:

- New Status
- Distance
- Timestamp of Transition
- Duration Since Previous State

This structured logging creates a clear audit trail that can be used to calculate cleaning response times, identify peak hours, and evaluate table turnover efficiency. *See appendix B*

3.1.4 Dashboard Interface

The Node-RED dashboard is accessible via any browser on the local network. Each table is represented by a responsive card displaying:

- Status: "Free," "Occupied," or "Needs Cleaning"
- Distance: Latest sensor reading
- Since: Timestamp of last state change
- Duration: Time elapsed in the current state

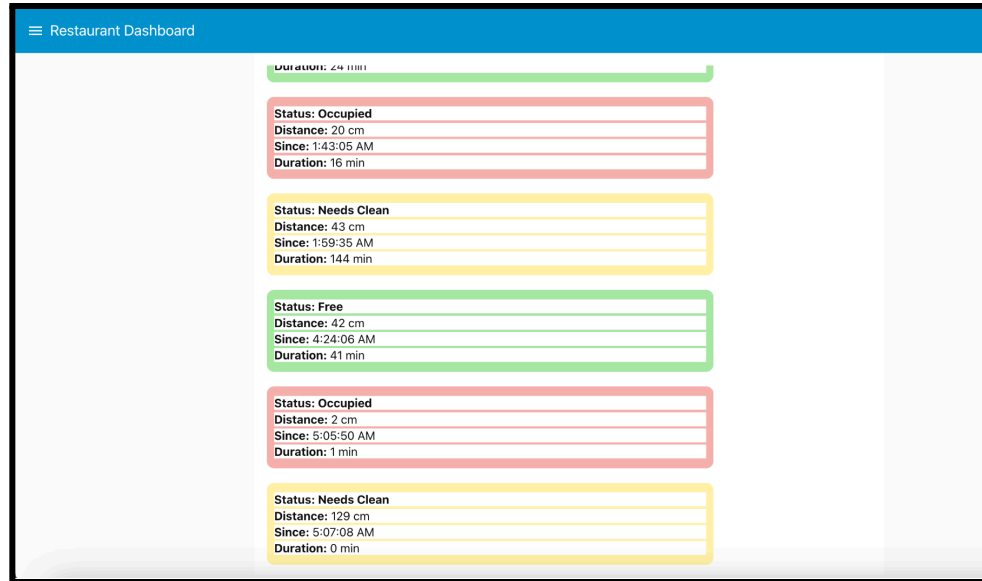


Figure 7. Table Management Dashboard Source: Author's own work.

Color-coded indicators are used to support fast recognition: green (Free), red (Needs Cleaning), and yellow (Occupied). The system was designed for use on a tablet or laptop placed at the host station, providing live feedback to staff.

3.1.5 Design Choices

Ultrasonic sensors were chosen due to their affordability, robustness, and ability to operate in various lighting conditions. Unlike visual systems (e.g., cameras), they offer a privacy-respecting and less intrusive alternative. An FSR (Force Sensitive Resistor) sensor was initially considered; however, it would be subject to faster wear and discomfort due to its placement on seating surfaces, making ultrasonic sensing more practical.

The division of labor between Arduino and Raspberry Pi ensured optimized performance: Arduino handled sensor reading at a high sampling rate, while Raspberry Pi executed more complex logic, managed network connections, and handled data visualization.

Node-RED was selected for its graphical flow interface, which simplifies real-time data orchestration. It also includes native dashboard nodes and integration with APIs and Google services, speeding up prototyping and customization.

3.2 Queue Monitoring System

This subsection outlines the development and deployment of a real-time queue tracking system that uses computer vision to monitor customer flow in front of a restaurant's ordering counter or buffet area. The system identifies when new people join the line and estimates their waiting time, updating a visual dashboard and logging results into Google Sheets. This section is divided into six parts: hardware configuration, software architecture, data-flow design, dashboard interface, rationale for key design choices, and known limitations.

3.2.1 Hardware Configuration

The queue monitoring system was deployed using a basic webcam positioned at torso height, approximately 1.5 meters away from the customer line and oriented perpendicularly to the queuing zone. This placement ensured minimal occlusion and allowed a clear view of the upper body (shirt area) for vision-based tracking. The camera used was a standard USB webcam operating at 720p and 30 frames per second, connected directly to a Raspberry Pi 4 for edge processing.

No sensors were needed beyond the camera. The simplicity of the setup made it cost-effective and easily deployable. For development and testing phases, an Apple MacBook Pro with an M1 chip was used as the processing unit before deployment on the Raspberry Pi.

3.2.2 Software Architecture

The software system was split into two coordinated components:

1. **Queue Detection Script (Python + MediaPipe):**

A Python script utilized MediaPipe's pose detection pipeline to identify people based on the upper-body landmarks (shoulders and hips). When the script detects a person standing within a predefined frame area (queue zone), it extracts color information from their shirt and pants to generate a unique profile. Instead of facial recognition, which poses privacy concerns, the system uses average RGB color from the torso area as a soft identifier.

If a new shirt+pants color profile does not match any in the known queue list (within a color similarity threshold), it is treated as a new customer entering the queue. The script assigns a person ID (e.g., "Person_001") and records their entry timestamp. When that profile disappears from the frame for a few seconds, the exit timestamp is logged, and wait time is computed.

All entries (ID, entry/exit time, wait duration, and shirt color in HEX) are saved in two formats:

- A final CSV file (queue_log_final.csv)
- A live JSON file (queue_live_data.json) for Node-RED to read.

2. **Node-RED Flow (Live Dashboard + Google Sheets Logging):**

Node-RED continuously reads the live JSON file and parses it into table data. A function node formats each row into a format compatible with Google Sheets (array of strings) and sends it to the GSheet node. Simultaneously, a dashboard table displays the data in real-time using the ui_table node. The logic also includes:

- An inject node triggering reads every 60 seconds.
- A file in node fetching the JSON file from the correct path.
- A json node parsing the content.
- A function node transforming it into spreadsheet rows.
- Output to both the dashboard and Google Sheets.

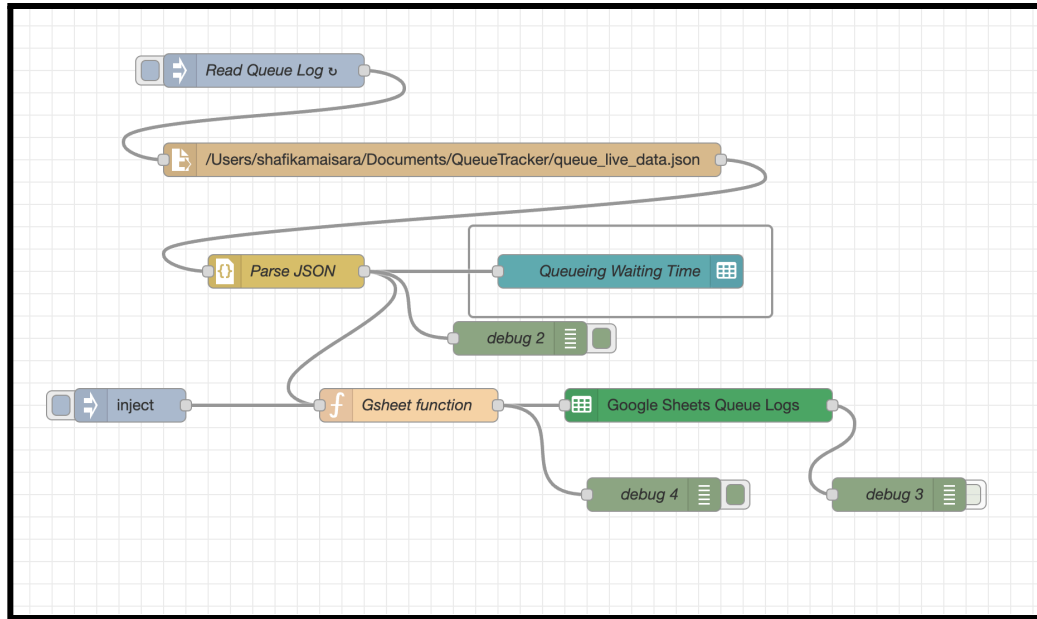


Figure 8. Queuing Monitoring Node Red Flow. Source: Author's own work.

3.2.3 Data-Flow Design

The system's data flow follows a hybrid pattern of file-based transfer and in-memory data processing: Camera Input → Python Pose Detection → Color Profile Matching → JSON Export → Node-RED File Input → Table Logic + Google Sheets Logging

Specifically:

- The Python script generates a live JSON log with structured customer entries, updating every time a person exits the frame.
- Node-RED polls this file using a timed inject node.
- A file in node reads /Users/shafikamaisara/Documents/QueueTracker/queue_live_data.json.
- Parsed JSON is sent to both a debug node (for monitoring) and a function node.
- The function node structures the log into a 2D array ([[ID, Entry Time, Exit Time, Wait Time, Shirt HEX]]) for compatibility with the Google Sheets API.
- The data is written to the "QueueLogsFixed" sheet using a GSheet append node.
- Both real-time display and historical records are maintained.

This design supports decoupling of acquisition and visualization, allowing the tracking script to run independently and the dashboard to fetch updates periodically.

3.2.4 Dashboard Interface

The interface is built using Node-RED's ui_table node and includes the following columns:

- Person ID

- Entry Time
- Exit Time
- Wait Time (seconds)
- Shirt Color (HEX)

Color coding helps staff distinguish queue duration visually. The table updates every few seconds, providing a near-real-time overview of queue activity. The dashboard is web-hosted by Node-RED and accessible over the local network, typically from a front-of-house tablet or staff workstation. This setup allows queue length and wait times to be monitored without interrupting ongoing service. Debug nodes are also included to monitor payload transformations, aiding in troubleshooting and system calibration.

3.2.5 Design Choices

Several technical and ethical factors influenced the system's design:

- Pose Detection over Face Detection: The use of torso and clothing color avoids collecting biometric data, supporting privacy-by-design principles.
- MediaPipe + RGB Matching: This combination enables low-compute, lightweight person detection without tracking facial features.
- Shirt Color as Soft Identifier: Offers a balance between anonymity and uniqueness needed for queue differentiation.
- JSON Live File Buffer: Decouples script and Node-RED system, allowing offline testing and easy integration.
- Node-RED Integration: Enables easy visualization, logging, and integration with Google Sheets without complex server-side code.

These design choices make the system lightweight, privacy-conscious, and adaptable to different environments.

3.3 Smart Buffet Monitoring System

This subsection describes a working prototype for monitoring a buffet tray's fill level and the surrounding environmental conditions. The objective was to provide kitchen staff with real-time information about when a tray is becoming empty and to issue alerts if temperature or humidity leave predefined safe ranges. The discussion that follows is organized into six parts: hardware configuration, software architecture, data-flow design, dashboard interface, rationale for key design choices, and known limitations.

3.3.1 Hardware Configuration

A commodity USB webcam was mounted directly above the buffet station, positioned approximately 1.2 meters above the serving surface. The camera's lens pointed straight downward so that the entire tray area was visible in each frame. The webcam provided a 640 × 480-pixel image at about 15

frames per second. This arrangement ensured consistent framing and minimized occlusion by diners' hands.

An Arduino Uno board was used to capture ambient temperature and humidity. A DHT20 sensor, connected via the Arduino's I²C pins (A4 for SDA, A5 for SCL), provided temperature readings with ± 0.3 °C accuracy and humidity readings with ± 2 % RH precision. Every two seconds, the Arduino sketch read temperature and printed a line of the form "Temperature = XX.X C" to its USB serial port. Immediately thereafter, it read humidity and printed a line of the form "Humidity = YY.Y %." The Arduino's USB connection supplied both 5 V power and a 9600 baud serial link to the adjacent Raspberry Pi.

A Raspberry Pi served as the central edge-compute node. It performed three roles simultaneously: first, it ran a Python script that captured frames from the webcam and classified tray fill level using a pre-trained convolutional neural network (CNN); second, it received serial data from the Arduino and forwarded raw lines for later combination; and third, it hosted a Node-RED instance on port 1880, which integrated incoming feeds, broadcast live updates via WebSocket, and stored history in Firebase Realtime Database. The Raspberry Pi also served a static HTML dashboard file so that any browser on the local network could view live status and historical charts. All hardware components were powered from a small UPS to tolerate brief power fluctuations.

3.3.2 Software Architecture

The software architecture comprises two primary components on the Raspberry Pi, a Python-based tray-classification and serial-forwarding script, and a Node-RED flow engine that processes incoming data. A separate Firebase Realtime Database instance provided cloud-based persistence, and a lightweight HTML/JavaScript dashboard consumed data via WebSocket and HTTP.

3.3.2.1 Tray-Classification and Serial-Forwarding Script

When the Raspberry Pi boots, it immediately starts a Python program responsible for two tasks: capturing and classifying images of the buffet tray, and relaying raw temperature and humidity readings from the Arduino. This single script uses a pre-trained neural network which was saved as the file `buffet_tray_model.h5` to determine whether the tray is full, medium, low, or empty. That model was trained earlier in Google Colab using Keras (TensorFlow's high-level API for neural networks), where a collection of overhead tray photographs was labeled and the network learned to distinguish among those four states.

Every five seconds, the Python script grabs one frame from the USB webcam, which is fixed about 1.5 meters above the buffet tray. Rather than process continuous video, it takes a single still image and resizes it so that it matches the input dimensions used during model training. Once the image is prepared, the script feeds it into the loaded neural network. The network returns scores for each of the four classes. To avoid spurious flickering such as a momentary glare or a diner's hand briefly covering part of the tray the script keeps a short history of the last three predictions. Only if at least two of those three agree does it accept that class as the tray's current status; otherwise, it waits for the next frame. In practice, this simple "two-out-of-three" rule prevents rapid, false state changes.

Once a stable label emerges (for example, "LOW"), the Python script notes the current UTC time (in ISO 8601 format) and pairs it with the label and a fixed identifier ("Tray 1"). It packages those three

pieces of information into a small data object and sends it to Node-RED via an HTTP POST to the local endpoint. If Node-RED confirms receipt, the script moves on; if the POST fails, because Node-RED hasn't started yet or due to a transient network glitch the script retries until it succeeds. This way, no single classification event is dropped.

3.3.2.2 Node-RED Workflows

Node-RED serves as the integration hub that processes inbound messages from the Python script, broadcasts live updates to the web dashboard, and saves all events to Firebase Realtime Database. Two parallel subflows handle tray-status messages and environmental readings, respectively.

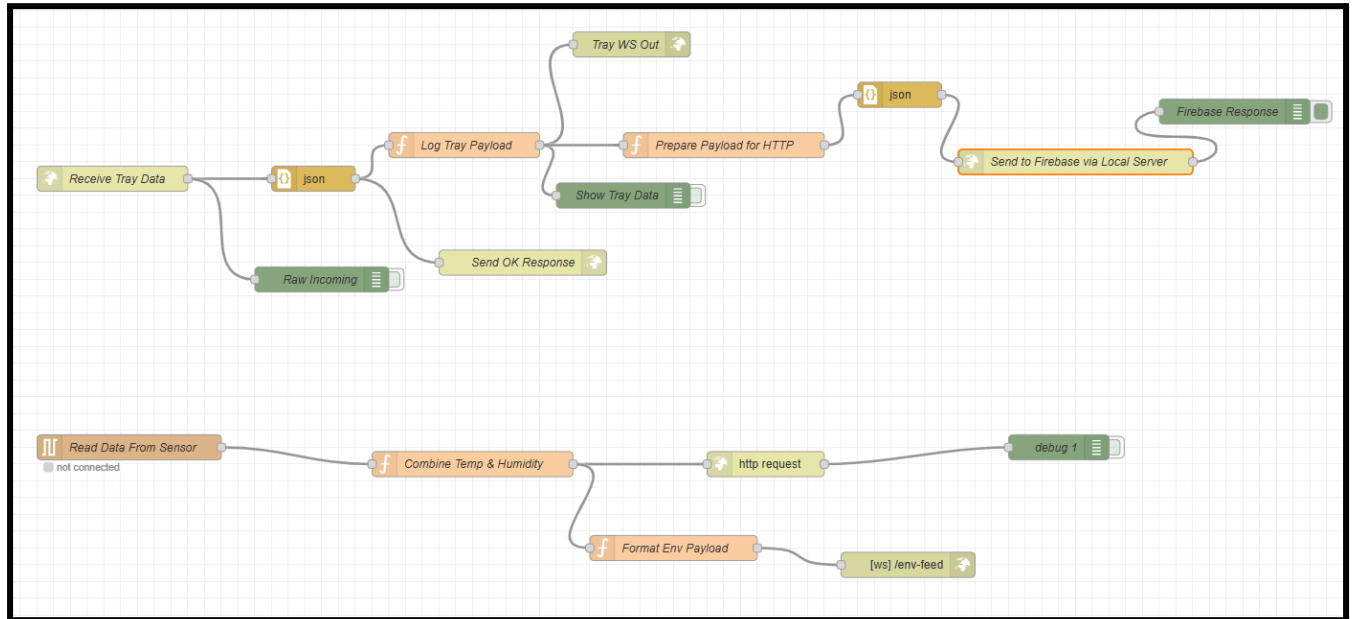


Figure 9. Node-RED Workflows: Two parallel subflows

1) Tray-Status Subflow

- First, Node-RED listens for HTTP POST requests at the endpoint `/tray-data`. Each such request carries a JSON object containing the tray identifier, a fill-level label, and a timestamp.
- Upon receipt, Node-RED immediately broadcasts the object to any connected web dashboards via a WebSocket channel named `/tray-feed`.
- Next, Node-RED reformats the same data so that the timestamp serves as a key under the path `/buffet_trays/Tray 1/history` in Firebase. This rewriting step produces a structure that appends each quick fill-level update in chronological order.

In effect, each time the Python script sends a tray status, Node-RED ensures two processes happen: the dashboard sees the update in real time, and the event is archived in Firebase under a timestamped history node.

2) Environmental Subflow

- Concurrently, Node-RED opens a WebSocket at /env-raw that receives raw serial lines forwarded by Python, each line being either a “Temperature = XX.X C” string or a “Humidity = YY.Y %” string.
- A dedicated function within Node-RED holds onto these lines until it has both a temperature value and a humidity value. Once the pair is available, Node-RED constructs a single JSON record that includes ambient temperature, ambient humidity, the current timestamp, boolean flags for threshold violations (such as “high_temp” if temperature exceeds 30 °C), and a concise human-readable status string.
- That combined message is then broadcast over a second WebSocket channel, /env-feed, so any open dashboards immediately receive the new readings and can update their displays.
- Finally, Node-RED reformats the record for Firebase, appending it under /env_data/Tray 1/history/<timestamp>. This ensures every two-second pair of sensor readings is archived for later analysis

Accordingly, Node-RED handles message parsing, WebSocket broadcasting, and Firebase persistence for both tray status and environmental data, keeping the Python script focused solely on acquisition and inference rather than on orchestration or storage.

By dividing responsibilities this way, the prototype achieves real-time responsiveness (through WebSocket broadcasts) and reliable history logging (through Firebase writes), while minimizing complexity in the Python script. The neural network file buffet_tray_model.h5 which was created in Google Colab serves its single purpose of classifying tray images, and all subsequent combination and persistence logic is neatly encapsulated within Node-RED.

3.3.3 Dashboard Interface

The dashboard provides a real-time interface for monitoring buffet tray status and ambient environmental conditions. It is implemented as a single HTML page with embedded JavaScript and CSS and is hosted locally on the Raspberry Pi. The Pi is used simply as a local server to run the required files and processes. Once running, the dashboard can be accessed from any browser on the same local network.

When the page loads, it opens two WebSocket connections:

- /tray-feed to receive tray classification updates
- /env-feed to receive temperature and humidity readings

The interface is structured into three visual tray blocks labeled T1, T2, and T3. Each block displays the most recent:

- Temperature reading (°C)
- Humidity reading (%)
- Tray status, based on image classification (e.g., “Empty”)

While the current prototype only processes input from a single tray, the dashboard can be developed to support multiple trays in the future. Each tray block updates independently based on its identifier, which is passed along with the data sent from the Python script and parsed in Node-RED.

When a tray is detected as empty, a red alert banner appears prominently below the tray blocks. The banner includes a warning message (e.g., “Tray 1 is empty – time to refill!”) and is styled to capture immediate attention through color and spacing, without interfering with the rest of the interface. This provides staff with a clear signal when action is needed.

The page layout is styled using CSS Flexbox to maintain a clean, consistent arrangement across screen sizes. Tray blocks use dark card styling with contrasting white text, while alerts are highlighted in red with soft shadows. Animations such as alert fade-in and blinking are implemented using standard CSS transitions. A “Connect” button in the header is available for manually reinitializing WebSocket connections during development or testing.

The dashboard updates continuously as new messages arrive from Node-RED. No user input or page reload is required, allowing the interface to run passively in the background. Overall, it provides an accessible and focused display that helps staff monitor the buffet area in real time and respond quickly to empty trays or unsafe environmental conditions.

3.4 System Integration

Although the buffet tray monitoring and table occupancy systems were developed separately during prototyping, their architecture and data structures were deliberately designed to be interoperable. Both subsystems rely on sensor input processed through a Raspberry Pi, which hosts Node-RED for real-time data routing, WebSocket broadcasting, and database storage. The consistent use of JSON message formats and modular flows enables the systems to be combined into a unified dashboard for operational oversight. This design reflects the cyber-physical systems (CPS) principle of modularity and distributed sensing, as emphasized by Oks et al. (2022), who highlight the importance of loosely coupled, interoperable modules in Industry 4.0 environments to enable scalable, resilient service delivery.

Both subsystems adopt a common logic: sensor-based input (ultrasonic for tables, camera and DHT sensor for buffet), real-time classification or measurement, and structured JSON outputs. These outputs are routed through Node-RED and either stored (e.g., Firebase or Google Sheets) or visualized through a dashboard. While each subsystem currently logs data independently, the architecture allows consolidation under a shared backend to support unified monitoring and decision-making. This architectural compatibility supports a seamless transition from isolated prototypes to an integrated product offering.

A plausible scenario is a peak-time university lunch service where all tables are nearly full and one buffet tray is detected as “Low.” The dashboard, accessible to both kitchen and floor staff, shows real-time tray fill status alongside table occupancy. The kitchen staff is prompted to prepare a refill while the service team anticipates upcoming table availability. As several tables approach typical meal duration thresholds, the staff can coordinate tray replenishment with guest turnover. This coordinated

action reduces idle guest waiting, avoids overproduction, and ensures timely replenishment , key operational efficiencies in a high-throughput environment.

The potential of this integrated setup extends into data-driven performance tracking. A range of combined KPIs could be extracted from both subsystems together, such as:

- **Refill Responsiveness:** Time between a “Low” detection and kitchen action, contextualized by table occupancy
- **Synchronized Demand Indicator:** Whether high occupancy correlates with refill events, measuring alignment between usage and kitchen behavior
- **Waste Risk Score:** Refill actions taken while occupancy is declining, indicating overproduction likelihood
- **Occupancy-to-Fill Lag Time:** Delay between expected peak use and actual tray replenishment

These KPIs move beyond isolated system monitoring and begin to quantify operational coordination which is a core aspect of digital service maturity. As Teichert (2023) notes, higher levels of digital transformation rely not just on sensing or automation but on the integration of data streams to support adaptive frontline decisions. By connecting table activity and buffet refill status, the system supports cross-functional responsiveness in real time.

While the prototype implementation stored buffet data in Firebase and table data in Google Sheets, future versions can merge these into a unified cloud database, enabling richer analytics and simplified maintenance. Furthermore, the shared dashboard can be expanded to allow dynamic filtering, KPI visualization, and role-specific views for kitchen, service, or management staff.

Overall, this integration demonstrates how well-designed modular CPS components can be unified into a coherent operational system that supports real-time action, longer-term planning, and a foundation for service innovation.

4. Discussion and Analysis

4.1 Operational Impact and Sustainability

The combined prototype of the buffet monitoring and table occupancy systems was developed to improve coordination between the kitchen and the dining area in real-time. While each subsystem focuses on a distinct aspect of the restaurant operation, together they form a basic cyber-physical system (CPS) designed to reduce food waste, improve staff responsiveness, and streamline service workflows.

Operational Efficiency and Coordination Gains

The table monitoring solution uses ultrasonic sensors to detect whether tables are occupied or available. It helps staff understand dining room dynamics, such as when turnover is slowing down or when queues are forming. Meanwhile, the buffet monitoring system uses a machine learning model to detect tray food levels and a sensor-based method to monitor environmental conditions. Together, these components improve awareness across the service line.

For instance, if the tray detection system classifies a tray as “low” and the table system indicates a high number of seated guests, kitchen staff can prioritize refilling without needing verbal communication. The real-time web dashboard, which aggregates tray, queue, and environment status, supports such decision-making by making relevant insights immediately visible to frontline staff.

Food Waste Reduction and Lean Principles

By reducing overproduction and enabling targeted refills, the buffet system helps prevent food waste. At the same time, the table monitoring tool minimizes unnecessary staff movement by showing table status without requiring physical checks. These benefits align with lean production principles like waste elimination and better timing of activities, which have been shown to improve sustainability in hospitality environments (Dutta et al., 2014). Instead of preparing food based on fixed assumptions, the system allows kitchens to respond to actual consumption patterns in near real time.

Service Quality and Responsiveness

The unified system also supports a smoother guest experience. Avoiding delays in refilling trays, reducing waiting times for tables, and providing staff with a clear view of the floor all contribute to better service delivery. The integration of environmental monitoring (e.g., temperature and humidity around buffet trays) adds an additional safety layer. As Luque et al. (2023) argue, such environmental sensing is especially important in food environments, where minor temperature deviations can impact quality and compliance.

Sustainability Benefits

From a sustainability perspective, the system contributes in three main areas:

- **Environmental:** Reduced food waste means less discarded food and lower energy use associated with overproduction.
- **Economic:** Improved staff efficiency and better use of resources (e.g., reduced overcooking) can lower operational costs over time.
- **Social:** Reducing stress on staff and creating a more organized working environment improves well-being and communication between kitchen and floor teams.

Scalability and Future Integration

Although the prototypes currently operate with separate data backends (Google Sheets and Firebase), they were designed with modularity in mind. A shared database and combined dashboard interface were identified as future improvements. In practice, integrating the systems fully could support more advanced coordination and even predictive analytics, such as forecasting when the next refill will be needed based on table occupancy and tray history.

Cost Consideration

For this prototype, we intentionally used low-cost components such as ultrasonic sensors, DHT22 temperature and humidity sensors, and a Raspberry Pi to validate functionality and system logic at a minimal cost. However, moving toward real-world implementation or commercialisation, particularly in environments with larger scale, multiple trays, and dozens of tables, the cost of components, installation, and maintenance would add up significantly. For instance, deploying 20–30 sensor units

across a large dining space would require additional investment in stable power supply, protective casing, and potentially industrial-grade sensors for durability.

Since our solution is inherently data-driven, cloud integration becomes a major cost factor. During development, we used Firebase's free Spark plan, which was sufficient for logging and visualising prototype data. However, a production system would require migrating to Firebase's Blaze plan, which operates on a pay-as-you-go model. This includes charges like \$5 per GB/month for Realtime Database storage and \$0.026 per GB/month for Cloud Storage, in addition to bandwidth and request fees (Firebase Pricing).

Other alternatives such as AWS IoT Core and Google Cloud Platform offer similar services, with AWS charging around \$1 per million messages for basic MQTT data handling and Google Cloud Storage starting at ~\$0.020 per GB/month depending on the region (AWS IoT Pricing, GCP Storage Pricing). Evaluating these options in terms of affordability, scalability, and support for analytics will be crucial in selecting a commercially viable backend.

4.2 Cross-Sector Transferability

What started as a system built specifically for buffet-style restaurant operations gradually evolved into a flexible solution that can be used across different sectors within the hospitality industry. Initially, our focus was on solving problems related to table turnover, queue congestion, and food waste in buffet settings. However, as the project progressed, we realized that the same principles could be applied in broader contexts such as hotels, food courts, and other hospitality service environments. These sectors face similar challenges in terms of managing flow, optimizing resources, and ensuring service quality which our system addresses through real-time monitoring, smart alerts, and historical data logging.

The digital system we have developed for smart restaurant operations. The focus on table management, queue monitoring, and buffet tray fill tracking might be originally designed for hospitality, but the digitalisation logic and structure behind it have much broader applications. These systems are modular, scalable, and relatively affordable, making them relevant not just for restaurants but also for production and manufacturing environments. The same concepts of tracking flow, visualizing live status, and logging data for analysis are key elements in modern industrial settings, particularly those adopting lean operations or looking to improve visibility. In fact, the same system logic used to manage restaurant tables and waiting lines can be applied to workstations and resource allocation in a production line, with minor adjustments. This is also in line with our earlier discussion in the business model section, where we pointed out that the core value of this solution lies not only in food service but also in enabling digital transformation for small- and medium-sized enterprises across different sectors.

Analogy Between Buffet Line and Manufacturing Flow

The buffet tray monitoring system shares a similar logic with a basic manufacturing flow. In restaurants, food trays are filled (input), consumed by customers (output), and need timely refilling (buffer management). The same applies in manufacturing where raw materials arrive, are held in WIP

(Work-in-Progress) buffers, and then processed and moved along. The visual fill detection system used in the buffet can easily be adapted to monitor inventory in bins, shelves, or pallets, with categories like "Full," "Half," or "Low." The core idea to detect the state, log it, visualize it, and alert when needed have the same intentions.

Digitalisation Logic as a Cross-Sector Enabler

What makes this transferable is the structure, not just the application. Sensor-based input collection, edge computing, local dashboards, and cloud logging form a backbone that works in any data-driven environment. In restaurants, this helps reduce wait times or monitor cleanliness. In factories, this same logic powers just-in-time production, lean scheduling, and OEE tracking. Our use of Node-RED to build custom flows can be replicated to build machine dashboards, production counters, or quality-check timers. Even the dashboard used for queue monitoring, built originally to count people and detect entry-exit time, can be used to monitor operator presence in safety zones, track mobile robots entering a specific area, or visualize when a workstation is overloaded.

Applying Monitoring Principles to Manufacturing Systems

The queueing system used a simple way to differentiate people based on color profiles without identifying them, which is important when privacy or regulation like GDPR applies. In a factory, this can be used to monitor workers' activity anonymously, ensuring productivity without violating their personal privacy. Similarly, the logic used in the table management system with the free, occupied, needs cleaning can be mapped directly to a workbench or machine. For example, when a machine finishes a task, it can go into a "needs inspection" state before being marked "available" again. All of this can be done using basic ultrasonic sensors, making it cost-effective for small-scale production environments where budget is limited.

Relevance to Other Production Systems

What makes this system stay relevant is that it's not tied to one use case. Whether it's lean production where you want to track how long each step takes, or warehouse logistics where you want to track when pallets enter or leave a zone, the modular nature of this system makes it adaptable. It can be deployed in:

- Production lines to track cycle time or station availability
- Warehouse zones to detect space usage and restocking cycles
- Logistics to monitor loading bay activity or storage buffer capacity
- Hospital food logistics to track tray delivery and patient wait times

By using basic hardware with smart flow design, this shows that cross-sector digitalisation doesn't have to be expensive or complex. One of the key enablers that ties all of these subsystems together is the data analytics capability. Through continuous data collection, integration, and logging, we can analyze patterns over time and generate insights that would otherwise go unnoticed. The ability to visualize and process live data not only improves current performance but also creates the foundation for predictive applications whether it's forecasting peak service hours in a restaurant or identifying

supply bottlenecks in a production line. This interconnected layer of analytics transforms isolated subsystems into an intelligent network, capable of learning and improving over time. The same logic applied to make a restaurant run smoother can be used to bring more control and visibility into manufacturing and logistics systems as well.

5. Conclusion and Reflection

This final chapter presents a synthesis of the project's outcomes, assesses the effectiveness of the implemented prototype, and reflects on both its limitations and potential for further development. It considers how the integrated system can support more efficient, responsive, and sustainable restaurant operations through data-driven insights and automation.

5.1 Key Outcomes and Impact

The project successfully demonstrated the feasibility and value of integrating lightweight digital systems into buffet-style restaurant operations. The prototype consisted of two main components: a buffet monitoring system and a table management module. These systems were independently developed but conceptually aligned to support visibility, coordination, and efficiency in food service environments.

The buffet monitoring module enabled real-time detection of tray depletion levels using a pre-trained convolutional neural network and humidity/temperature sensors to detect unsafe food conditions. The integration with Node-RED and Firebase allowed for smooth data processing, live dashboard updates, and historical logging. The system minimized guesswork and allowed for timely refills, directly supporting food safety and waste reduction goals.

The table and queue monitoring system complemented the buffet component by tracking customer flow and table turnover, which helped model demand patterns and supported better resource allocation. Together, the systems created a data-driven workflow that helps staff respond to food shortages, optimize table usage, and reduce customer wait times. This dual solution also laid the groundwork for creating combined operational KPIs such as average tray depletion time correlated with seating rate which is offering new strategic insights into service demand and kitchen preparation cycles.

The project emphasized how Cyber-Physical Systems (CPS) and lightweight edge computing can be applied in hospitality to promote sustainability and service quality. It aligned well with the lean principles covered in earlier chapters by reducing process waste (both food and time), automating key decisions, and enabling real-time feedback. The hands-on implementation also helped us develop practical skills in sensor integration, Python scripting, data processing with Node-RED, and designing intuitive dashboards, all within the constraints of real-world restaurant conditions.

While this prototype was developed on a small scale, it provides a scalable foundation for future commercial applications. It highlights the importance of cross-system data coordination and sets a clear direction for how digitalisation can meaningfully contribute to sustainable, efficient food service.

5.2 Limitations and Improvement Areas

5.2.1 Smart Buffet Monitoring System

While the buffet monitoring system achieved stable performance in tracking food levels and environment conditions, several limitations were encountered. Initially, a load-cell based weighing approach was considered to track tray fill levels. However, due to inconsistent and noisy readings influenced by surface placement and vibrations, this method was replaced with a vision-based classification approach. The camera system offered greater flexibility but introduced challenges in terms of lighting variation and occasional occlusion during heavy customer flow.

Although the image classification using a neural network provided reliable results, the model still occasionally struggled with borderline cases such as distinguishing between medium and low fill levels. The model was trained on a modest dataset (~800 images), and accuracy could improve with further data collection and augmentation. Additionally, the vision system currently requires a stable overhead setup, which may not be practical in all buffet settings without structural adjustment.

The environmental sensing using the DHT20 module worked reliably for this prototype. However, sensor placement was critical, readings were sometimes skewed by proximity to heat lamps or draft-prone locations. While the sensors provided valuable feedback on food safety thresholds, future versions could benefit from integrating multiple sensors per tray or using higher-resolution thermal imaging to map tray-level conditions more precisely.

To further enhance system utility, the following improvements are proposed:

- Expand the image training dataset for improved model accuracy.
- Implement confidence-based alert logic to reduce false positives.
- Improve dashboard design for mobile-first usage and role-based access (e.g., for kitchen vs. service staff).

5.2.2 Smart Table and Queue Monitoring System

While both the Smart Table Management and Queue Monitoring systems have worked well during testing and small-scale deployment, there are still a few challenges we noticed along the way. For the table management system, there were occasional false positives such as misplaced seats that ended up being detected as someone sitting. The setup also depends on fixed geometry, meaning if the table is moved or the seating layout changes, the sensor needs to be manually recalibrated. On the software side, the serial communication between Arduino and Raspberry Pi can sometimes be unstable after a reboot, causing the serial port path to reset in Node-RED. Also, because these sensors are exposed in public settings, they can be easily damaged or obstructed by things like spilled drinks or gum under the table.

On the queue monitoring side, the system is based on pose estimation and soft color identifiers, which means it struggles under certain lighting conditions or when people overlap in the frame. Without hard identifiers, someone walking out and returning quickly might be logged twice. And because the Python script communicates with Node-RED through a shared JSON file, timing issues can cause delays or misreads if the file isn't written or read properly. As a prototype, it works, but scaling this up would need more robust architecture and logic.

To address these limitations and prepare for future use cases, the following improvements are proposed:

- Expand the training dataset: Add more examples of people in different lighting, clothing, and movement styles to improve model accuracy in various conditions.
- Add confidence-based alert logic: Use threshold values based on detection confidence to reduce the number of false alerts and improve system reliability.
- Redesign dashboard for mobile and role-specific use: Optimize the interface for tablets and smartphones, and create separate views for different user roles (e.g., service staff vs. kitchen staff).
- Improve sensor protection: Use protective casings or enclosures to make the ultrasonic sensors more resistant to damage from public misuse or environmental factors.
- Integrate multi-sensor fusion: Combine data from additional sensors (e.g., thermal, pressure) to improve detection reliability and reduce false readings.
- Upgrade from file-based to real-time communication: Replace JSON file exchanges with MQTT or WebSocket protocols for better stability and scalability.
- Enable predictive analytics and insights: Start leveraging the collected data to predict peak periods, optimize resource allocation, and guide decisions based on trends over time.

5.3 Conclusion

To conclude the report, we have answered the research questions that we have

How can digital system solutions be implemented in a restaurant setting to improve operational visibility and efficiency?

Digital systems can be implemented in restaurants by integrating real-time monitoring tools into core service areas such as table management, queue tracking, and buffet refilling. Using technologies like ultrasonic sensors, computer vision, and edge computing (e.g., Raspberry Pi with Node-RED), restaurants can automate the detection of table occupancy, customer flow, and tray depletion. These modules feed into dashboards that provide live updates, allowing staff to make faster and more informed decisions. This improves table turnover, shortens wait times, and supports more coordinated operations between the kitchen and floor staff.

What are the key design considerations in selecting sensors and data flow systems for queue and table monitoring?

Sensor selection must balance cost, accuracy, durability, and privacy. Ultrasonic sensors were chosen for table monitoring due to their affordability and effectiveness in detecting customer presence without

intruding on privacy. For queue tracking, vision-based pose estimation using MediaPipe and shirt color detection was used instead of facial recognition to ensure GDPR compliance. On the data side, systems like Node-RED were implemented for their modularity and ease of integrating serial, JSON, and visual inputs into dashboards and cloud storage like Google Sheets or Firebase. These choices ensured lightweight, scalable, and privacy-conscious monitoring.

How can digital data from CPS modules be used to inform service strategies and reduce food waste in real-time?

CPS modules generate continuous data streams on tray fill levels, table usage, and queue lengths, which are visualized in dashboards to support real-time decision-making. For example, alerts based on tray depletion and customer density help kitchen staff refill only when necessary, reducing overproduction and waste. Data logs also enable performance metrics like refill responsiveness and table turnover time, guiding staff allocation and planning. Over time, this data supports strategic improvements by identifying patterns in demand and inefficiencies, enabling predictive service adjustments and leaner operations.

References

Buhalis, D., & Leung, R. (2018). Smart hospitality—Interconnectivity and interoperability towards an ecosystem. *International Journal of Hospitality Management*, 71, 41–50.

<https://doi.org/10.1016/j.ijhm.2017.12.002>

Cavoukian, A. (2010). *Privacy by Design: The 7 foundational principles*. Information and Privacy Commissioner of Ontario.

<https://www.ipc.on.ca/wp-content/uploads/resources/7foundationalprinciples.pdf>

European Union. (2016). *General Data Protection Regulation (GDPR)*. *Official Journal of the European Union*, Regulation (EU) 2016/679.

Eurostat. (2023). *Food waste in the EU: Statistics explained*. European Commission.

https://ec.europa.eu/eurostat/statistics-explained/index.php?title=Food_waste_in_the_EU

Jones, P., Hillier, D., & Comfort, D. (2008). Sustainability in the hospitality industry: Some personal reflections on corporate challenges and research agendas. *International Journal of Contemporary Hospitality Management*, 20(1), 7–13. <https://doi.org/10.1108/09596110810852128>

Jones, P., Hillier, D., & Comfort, D. (2014). Sustainability in the global hotel industry. *International Journal of Contemporary Hospitality Management*, 26(1), 5–17.

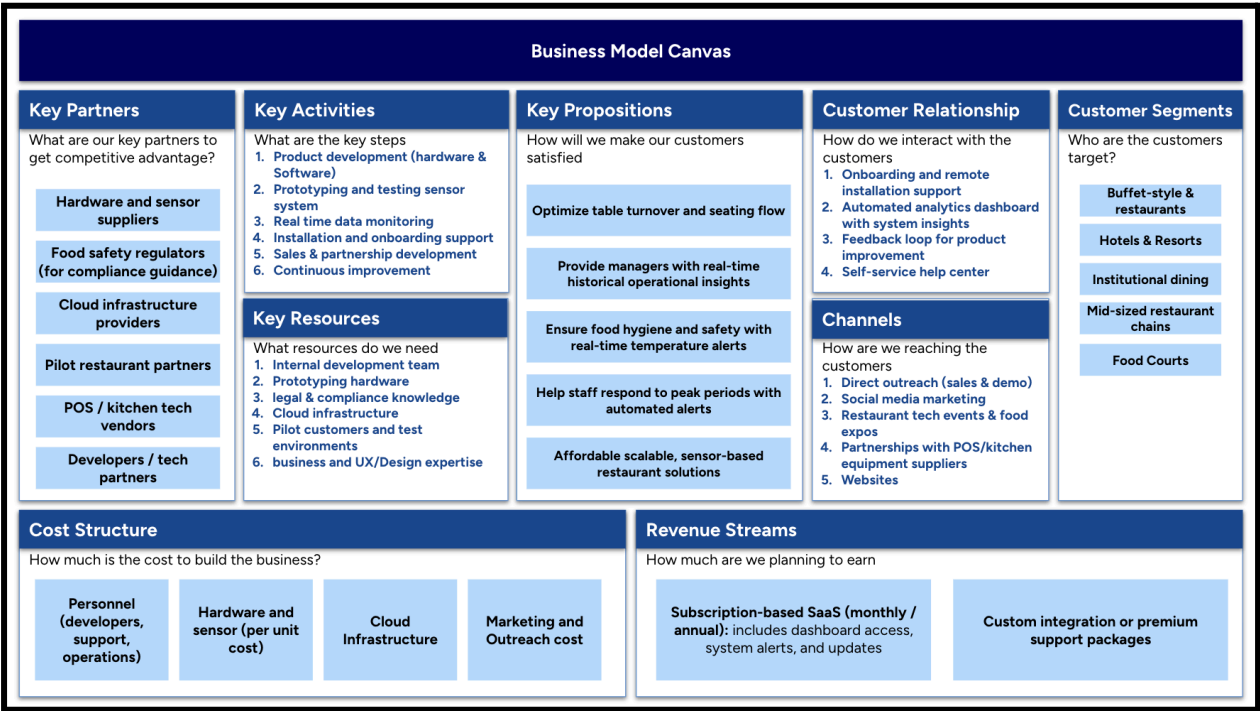
<https://doi.org/10.1108/IJCHM-10-2012-0171>

Lee, E. A., Bagheri, B., & Jin, C. (2015). Introduction to cyber-physical systems. *Proceedings of the IEEE*, 103(1), 5–7. <https://doi.org/10.1109/JPROC.2014.2373972>

- Lisitsyn, I., Chernukha, Y., Kotenkova, P., Kulikovskiy, V., & Bychkova, D. (2022). Cyber-physical systems in food production chain. *Procedia Computer Science*, 213, 2101–2107. <https://doi.org/10.1016/j.procs.2022.11.283>
- Monostori, L. (2014). Cyber-physical production systems: Roots, expectations and R&D challenges. *Procedia CIRP*, 17, 9–13. <https://doi.org/10.1016/j.procir.2014.03.115>
- Monostori, L., Kádár, B., Bauernhansl, T., Kondoh, S., Kumara, S., Reinhart, G., Sauer, O., Schuh, G., Sihn, W., & Ueda, K. (2016). Cyber-physical systems in manufacturing. *CIRP Annals*, 65(2), 621–641. <https://doi.org/10.1016/j.cirp.2016.06.005>
- Panwar, H., Kaushik, S., Saini, H., & Verma, S. (2019). Smart restaurant management system using IoT. *International Journal of Engineering and Advanced Technology*, 9(1), 7384–7389. <https://doi.org/10.35940/ijeat.A3096.109119>
- Papargyropoulou, E., Lozano, R., Steinberger, J. K., Wright, N., & Ujang, Z. B. (2014). The food waste hierarchy as a framework for the management of food surplus and food waste. *Journal of Cleaner Production*, 76, 106–115. <https://doi.org/10.1016/j.jclepro.2014.04.020>
- Rossini, M., Costa, C., Tortorella, F., & Portioli-Staudacher, S. M. M. (2019). The interrelation between Industry 4.0 and lean production: An empirical study. *Procedia Manufacturing*, 38, 844–851. <https://doi.org/10.1016/j.promfg.2020.01.119>
- Schuh, G., Anderl, R., Gausemeier, J., Wahlster, W., & Ten Hompel, M. (2017). *Industrie 4.0 Maturity Index: Managing the Digital Transformation of Companies*. Acatech STUDY. Herbert Utz Verlag. <https://en.acatech.de/publication/industrie-4-0-maturity-index-managing-the-digital-transformation-of-companies/>
- Silvennoinen, K., Heikkilä, L., Katajajuuri, J. M., & Reinikainen, A. (2015). Food waste volume and origin: Case studies in the Finnish food service sector. *Waste Management*, 46, 140–145. <https://doi.org/10.1016/j.wasman.2015.09.018>
- Spiekermann, S., & Cranor, L. F. (2009). Engineering privacy. *IEEE Transactions on Software Engineering*, 35(1), 67–82. <https://doi.org/10.1109/TSE.2008.88>
- Teichert, R. (2023). Digital transformation maturity in service systems: A review. *The Service Industries Journal*, 43(1–2), 120–147. <https://doi.org/10.1080/02642069.2022.2091161>
- Tornatzky, R., & Fleischer, M. (1990). *The processes of technological innovation*. Lexington Books.
- van den Hoven, J., Vermaas, P. E., & van de Poel, I. (Eds.). (2015). *Handbook of ethics, values, and technological design*. Springer. <https://doi.org/10.1007/978-94-007-6970-0>
- Womack, J., & Jones, D. (2003). *Lean thinking: Banish waste and create wealth in your corporation* (2nd ed.). Free Press.

Appendices

Appendix A: Business Model Canvas



Appendix B: Smart Table Management and Queueing Monitoring: Googlesheets Database Sample

docs.google.com/spreadsheets/d/13nv6CIT-Z6i9Pb-y30kelPnrhQZf00ZGF1JLPniYfRQ/edit?gid=2109176701#gid=2109176701

Restaurant Logs

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O
1	Status	Distance	Since	Duration											
2	Needs Clean	164 cm	10:14:23	0 min											
3	Free	-1 cm	10:14:46	30 min											
4	Occupied	1 cm	10:44:58	2 min											
5	Needs Clean	165 cm	10:47:03	1 min											
6	Free	165 cm	10:48:17	3 min											
7	Occupied	2 cm	10:51:56	1 min											
8	Needs Clean	351 cm	10:53:45	0 min											
9	Free	351 cm	10:53:57	4 min											
10	Occupied	21 cm	10:58:41	2 min											
11	Needs Clean	156 cm	11:00:46	0 min											
12	Free	156 cm	11:01:25	4 min											
13	Occupied	3 cm	11:05:30	1 min											
14	Needs Clean	198 cm	11:06:59	0 min											
15	Free	174 cm	11:07:36	3 min											
16	Occupied	3 cm	11:11:15	1 min											
17	Needs Clean	74 cm	11:12:58	0 min											
18	Free	75 cm	11:13:07	4 min											
19	Occupied	3 cm	11:17:51	1 min											
20	Needs Clean	148 cm	11:19:31	1 min											
21	Free	6 cm	11:21:04	3 min											
22	Occupied	2 cm	11:24:56	1 min											
23	Needs Clean	0 cm	11:26:33	0 min											

Convert to table

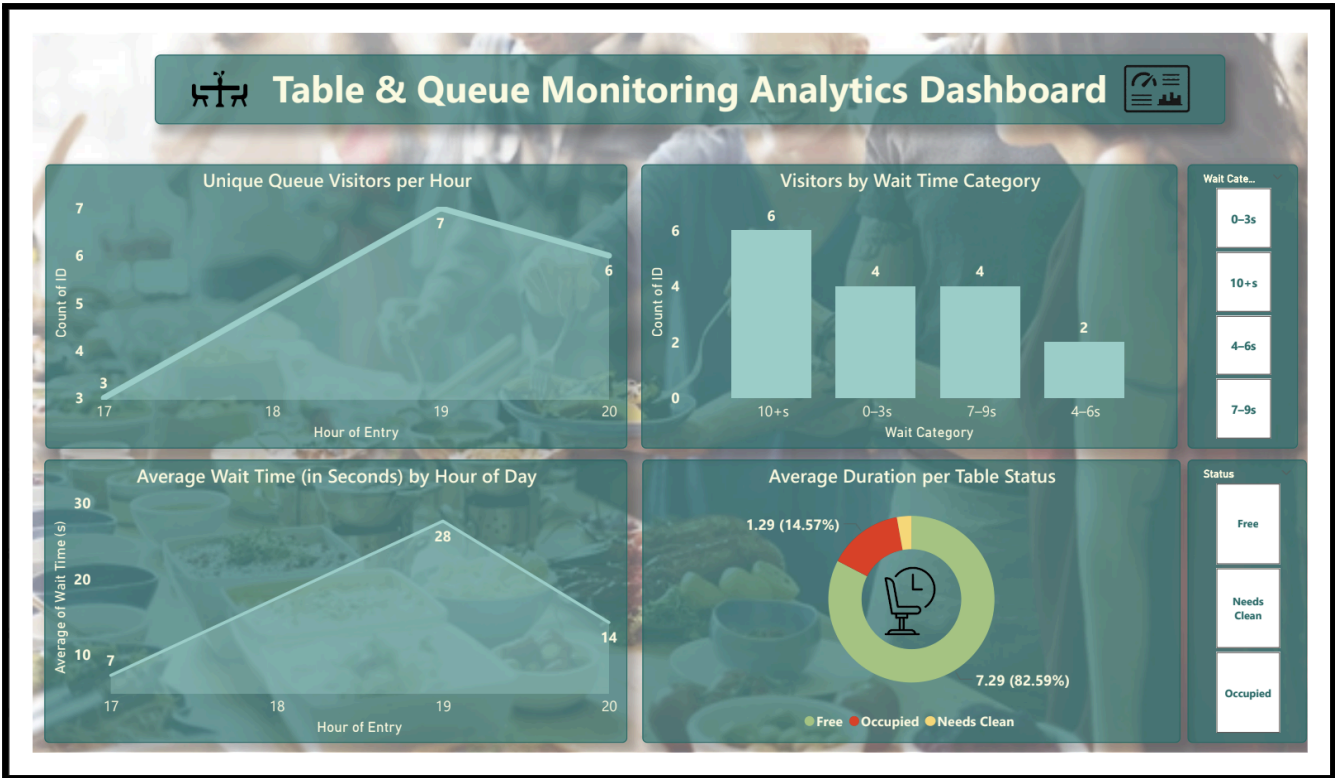
TableManagementRaspPi QueueLogsFixed

docs.google.com/spreadsheets/d/13nv6CIT-Z6i9Pb-y30kelPNrhqZfO0ZGF1JLPniYFRQ/edit?gid=1881601234#gid=1881601234

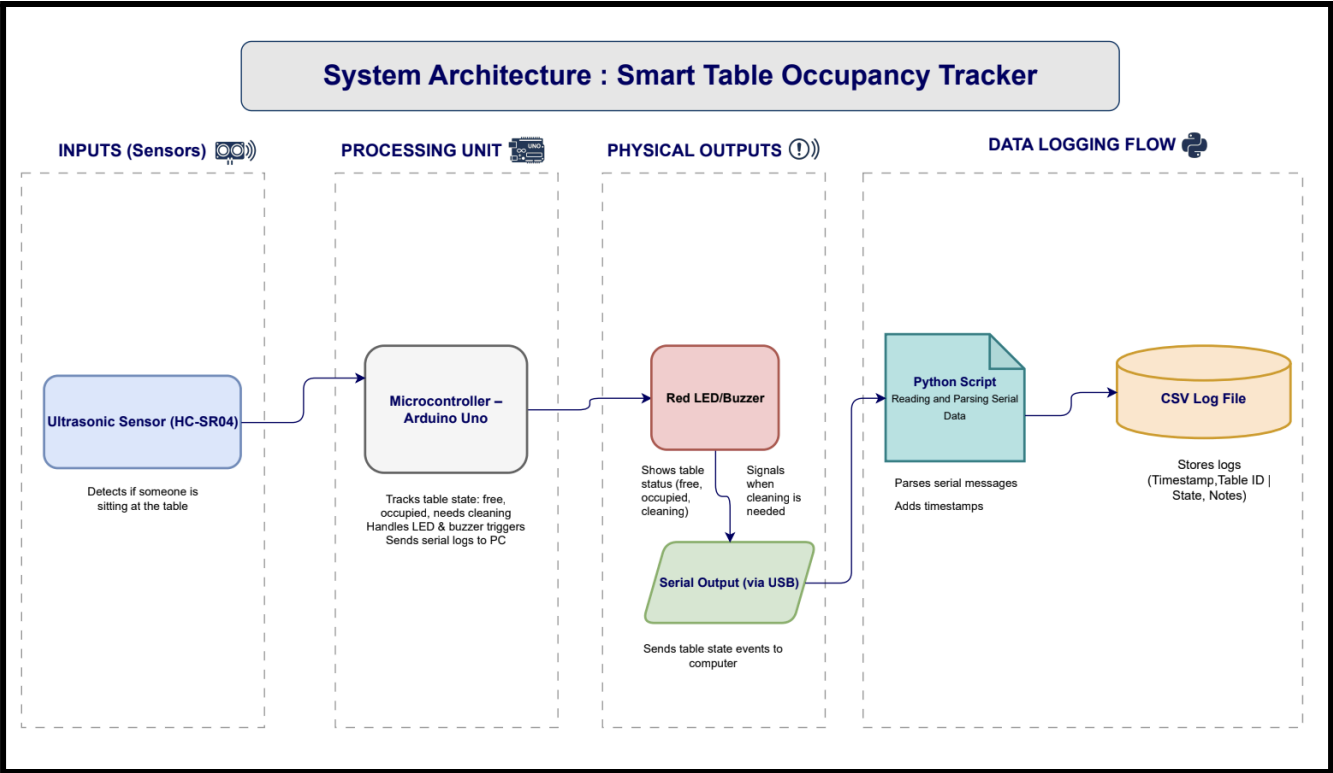
Restaurant Logs

ID	Entry Time	Exit Time	Wait Time (s)	Shirt Color (HEX)
Person_003	19:56:10	19:56:48	37.8	#535d7e
Person_004	19:56:12	19:56:48	36.1	#5773a2
Person_005	19:56:13	19:56:48	35.1	#7c93c0
Person_006	19:56:43	19:56:48	5.2	#98aad9
Person_007	19:56:45	19:56:48	2.7	#757680
Person_001	20:13:34	20:13:49	14.8	#5169a0
Person_002	20:13:35	20:13:49	14	#3c4a79
Person_003	20:13:37	20:13:49	12	#617ebf
Person_004	20:13:37	20:13:49	11.3	#817d81
Person_005	20:13:39	20:13:49	9.3	#b0a4a5
Person_006	20:13:42	20:13:49	6.6	#1f1129
Person_001	20:13:34	20:13:49	14.8	#5169a0
Person_002	20:13:35	20:13:49	14	#3c4a79
Person_003	20:13:37	20:13:49	12	#617ebf
Person_004	20:13:37	20:13:49	11.3	#817d81
Person_005	20:13:39	20:13:49	9.3	#b0a4a5
Person_006	20:13:42	20:13:49	6.6	#1f1129
Person_001	20:13:34	20:13:49	14.8	#5169a0
Person_002	20:13:35	20:13:49	14	#3c4a79
Person_003	20:13:37	20:13:49	12	#617ebf
Person_004	20:13:37	20:13:49	11.3	#817d81
Person_005	20:13:39	20:13:49	9.3	#b0a4a5
Person_006	20:13:42	20:13:49	6.6	#1f1129
Person_001	20:13:34	20:13:49	14.8	#5169a0
Person_002	20:13:35	20:13:49	14	#3c4a79
Person_003	20:13:37	20:13:49	12	#617ebf
Person_004	20:13:37	20:13:49	11.3	#817d81

Appendix C: Smart Table & Queue Monitoring Analytics Dashboard: Power BI Visualizations

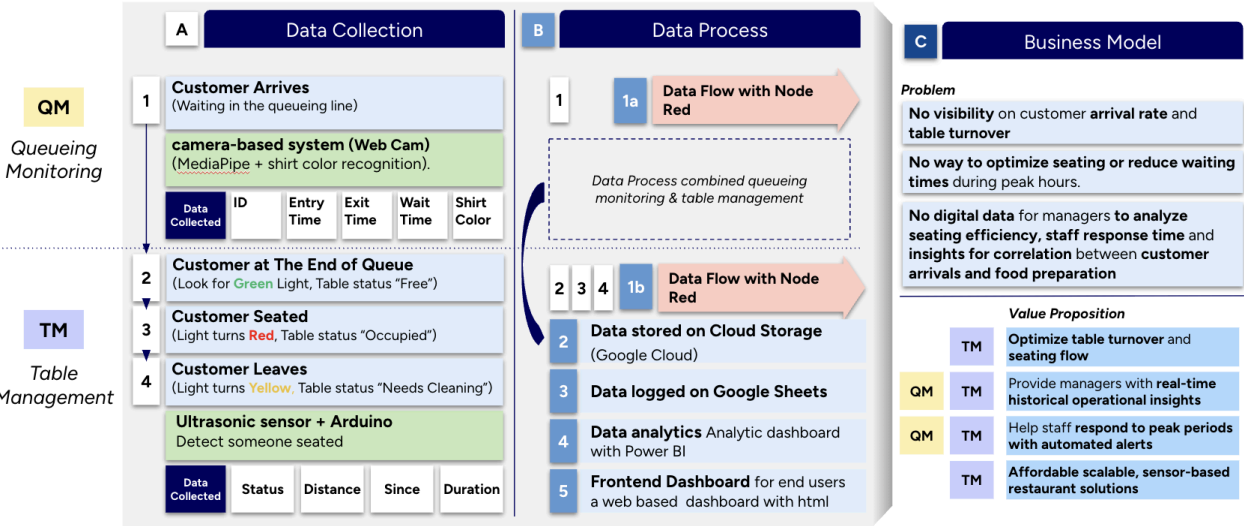


Appendix D: Initial Smart Table Management System Architecture Design

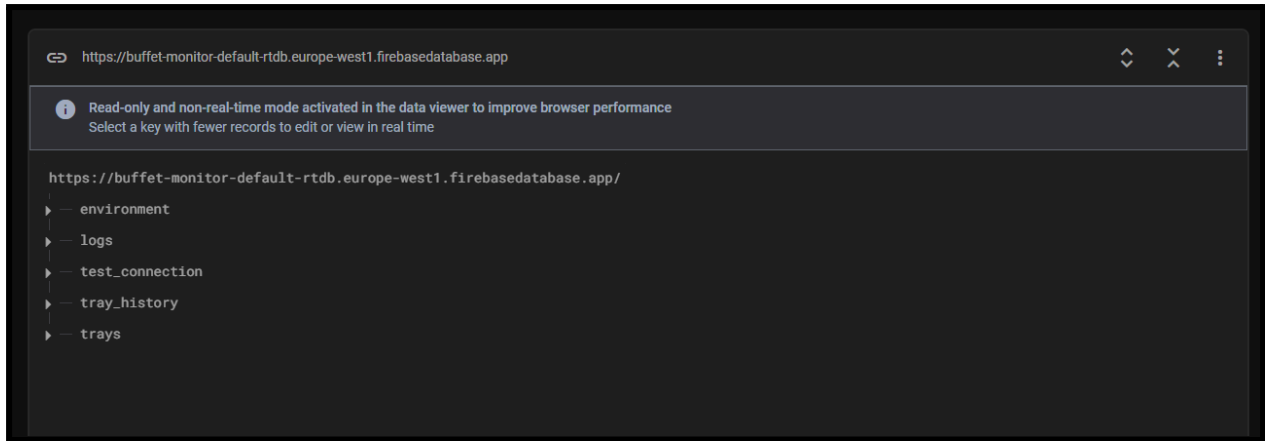
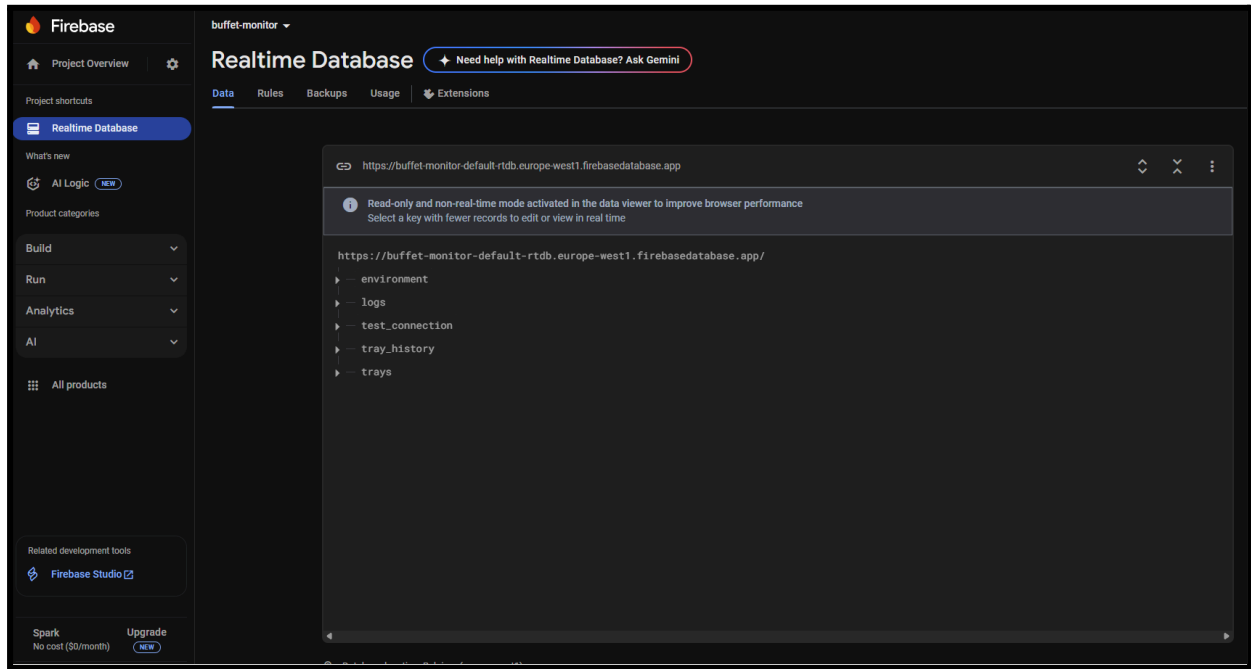


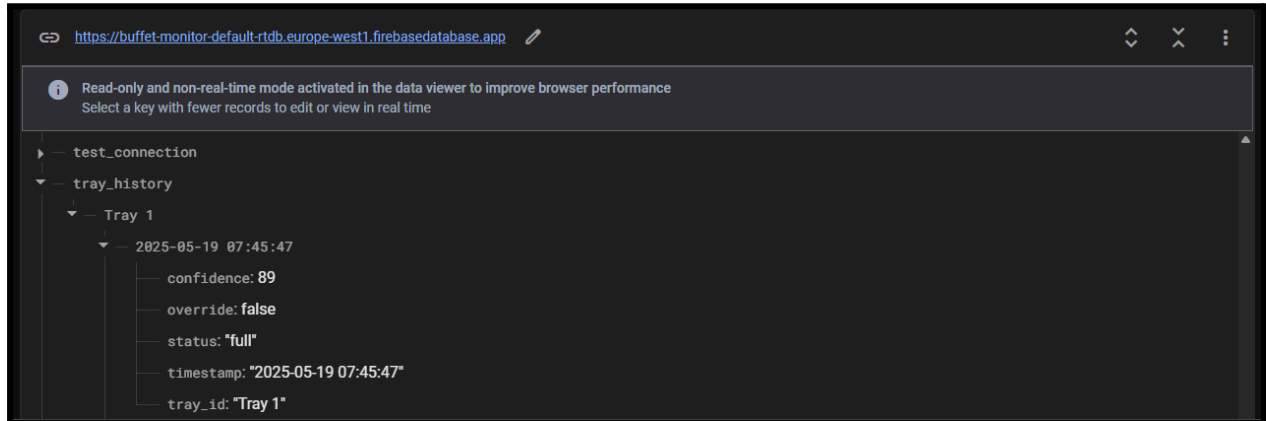
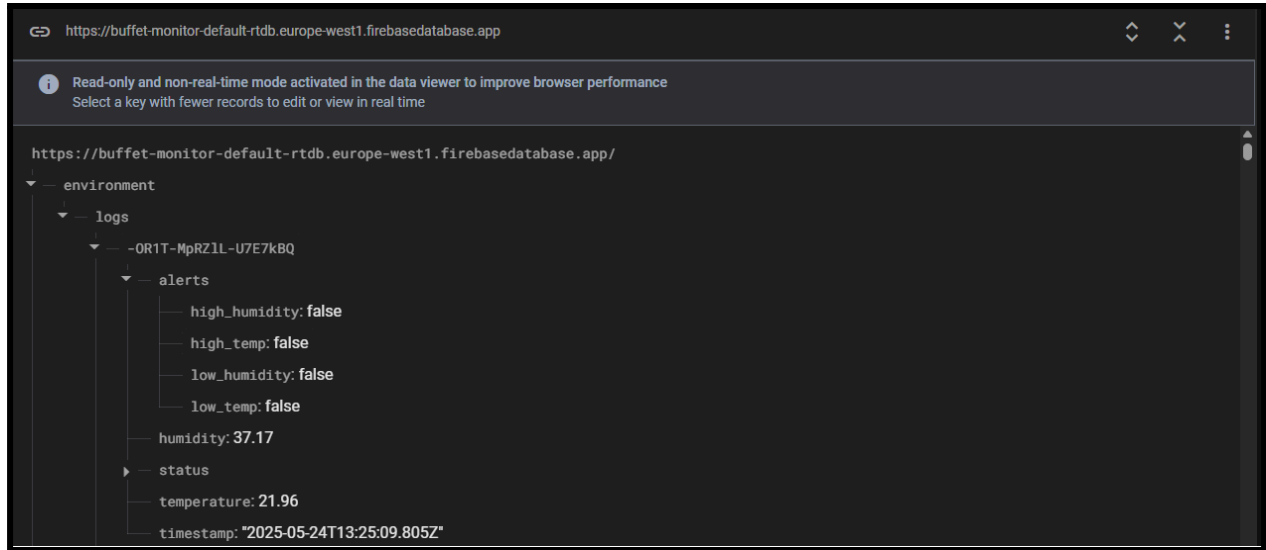
Appendix E: Final Smart Table Management and Queueing Monitoring System Architecture Design

Digital Solution for Restaurant Operations

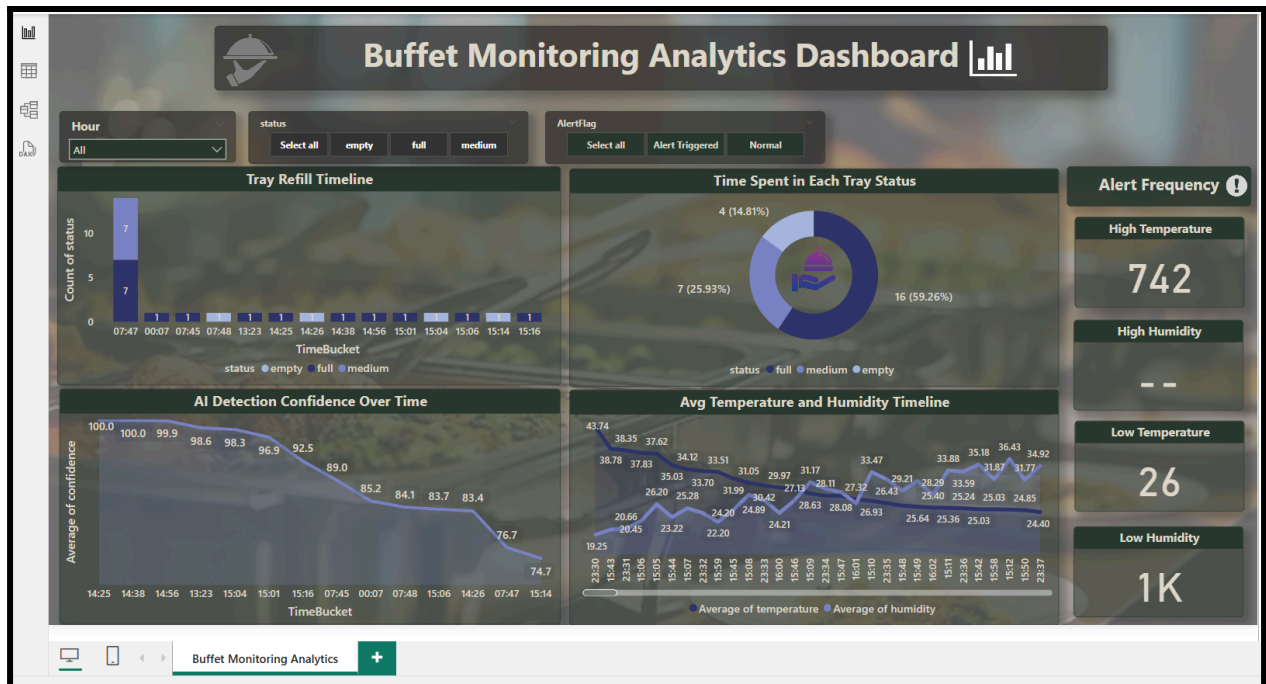


Appendix F: Smart Buffet Monitoring: Firebase Database Structure Samples





Appendix G: Smart Buffet Monitoring Analytics Dashboard: Power BI Visualizations



Appendix H: Smart Buffet Monitoring Analytics Dashboard: Power BI Data Model

Name: logs_output

Manage relationships | New measure | Quick measure | New column | New table | Mark as date table

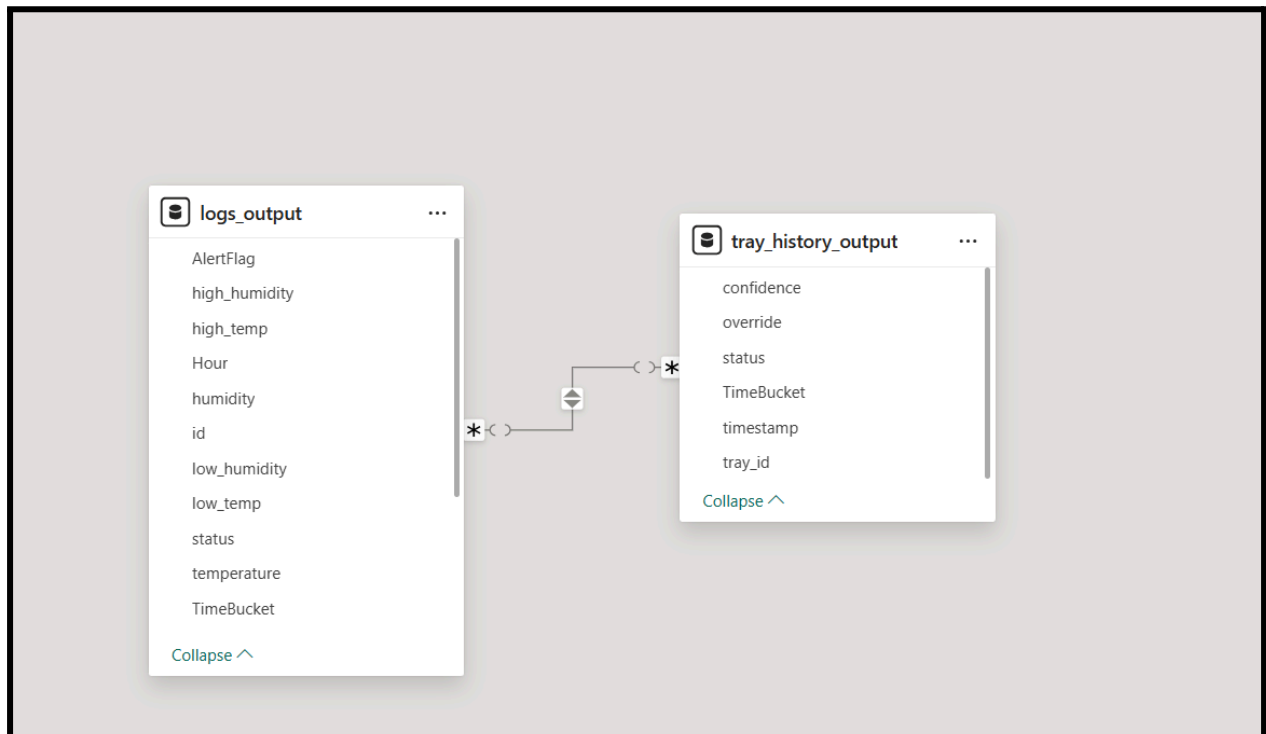
id	timestamp	temperature	humidity	status	high_temp	low_temp	high_humidity	low_humidity	tray_id	TimeBucket	Hour	AlertFlag
-OR2zWfoQ_NVbexrf-OA	5/24/2025 10:31:12 PM	21.45	39.49	OK	False	False	False	False	Tray 1	22:31	22	Normal
-OR2zWf6irDj5K1C-TW	5/24/2025 10:31:12 PM	21.45	39.49	OK	False	False	False	False	Tray 1	22:31	22	Normal
-OR2zJakv2X3gzloyDa	5/24/2025 10:31:30 PM	21.45	38.83	OK	False	False	False	False	Tray 1	22:31	22	Normal
-OR2zJlb_vNY3yrvVp	5/24/2025 10:31:30 PM	21.45	38.79	OK	False	False	False	False	Tray 1	22:31	22	Normal
-OR2zJ5k9CAa61EfmM	5/24/2025 10:31:55 PM	21.45	41.64	OK	False	False	False	False	Tray 1	22:31	22	Normal
-OR2zJLLXZ4uGvWmVqz	5/24/2025 10:31:55 PM	21.45	41.45	OK	False	False	False	False	Tray 1	22:31	22	Normal
-OR2zhDok8dXT_J3kgXD	5/24/2025 10:32:01 PM	21.45	41.08	OK	False	False	False	False	Tray 1	22:32	22	Normal
-OR2zhE9nDIZ6AipY3	5/24/2025 10:32:01 PM	21.45	40.71	OK	False	False	False	False	Tray 1	22:32	22	Normal
-OR2zhz9TGBtzpRbwEH	5/24/2025 10:32:03 PM	21.45	40.71	OK	False	False	False	False	Tray 1	22:32	22	Normal
-OR2zhjDJaGksU5S15	5/24/2025 10:32:03 PM	21.45	40.35	OK	False	False	False	False	Tray 1	22:32	22	Normal
-OR2zjzpZRJw0I9olxEC	5/24/2025 10:32:11 PM	21.45	39.55	OK	False	False	False	False	Tray 1	22:32	22	Normal
-OR2zj59h8OWIGPo_g6	5/24/2025 10:32:11 PM	21.45	39.40	OK	False	False	False	False	Tray 1	22:32	22	Normal
-OR3-WkuH9NTW7j54D3	5/24/2025 10:35:36 PM	21.45	38.88	OK	False	False	False	False	Tray 1	22:35	22	Normal
-OR3-Wl6xOEYCKReuH20	5/24/2025 10:35:36 PM	21.45	38.86	OK	False	False	False	False	Tray 1	22:35	22	Normal
-OR3-bFPK6E-QNkb-hA	5/24/2025 10:35:59 PM	21.45	38.91	OK	False	False	False	False	Tray 1	22:35	22	Normal
-OR3-bFfgVnvZ6qJyQk	5/24/2025 10:35:59 PM	21.45	38.86	OK	False	False	False	False	Tray 1	22:35	22	Normal
-OR3-cFK3ijZYCA76VC	5/24/2025 10:36:03 PM	21.45	38.82	OK	False	False	False	False	Tray 1	22:36	22	Normal
-OR3-cFia0wL1MYtQtd	5/24/2025 10:36:03 PM	21.45	38.83	OK	False	False	False	False	Tray 1	22:36	22	Normal
-OR3-dkOO1-sOOrkaJC1	5/24/2025 10:36:09 PM	21.45	38.82	OK	False	False	False	False	Tray 1	22:36	22	Normal
-OR3-dkgEb1vwbPwK3p	5/24/2025 10:36:09 PM	21.45	38.81	OK	False	False	False	False	Tray 1	22:36	22	Normal
-OR3-gk-yXapBzmK46	5/24/2025 10:36:21 PM	21.45	38.81	OK	False	False	False	False	Tray 1	22:36	22	Normal
-OR3-gkLwN_uPpU_V3t	5/24/2025 10:36:21 PM	21.45	38.82	OK	False	False	False	False	Tray 1	22:36	22	Normal
-OR3-hF2KTC2RdW2qLM	5/24/2025 10:36:23 PM	21.45	38.82	OK	False	False	False	False	Tray 1	22:36	22	Normal
-OR3-hFkmhYC8vNwSrs	5/24/2025 10:36:23 PM	21.45	38.81	OK	False	False	False	False	Tray 1	22:36	22	Normal
-OR3-EqameIFuHIGd4U	5/24/2025 10:36:27 PM	21.45	38.80	OK	False	False	False	False	Tray 1	22:36	22	Normal
-OR3-IfZUSVYDu5dLT	5/24/2025 10:36:27 PM	21.45	38.86	OK	False	False	False	False	Tray 1	22:36	22	Normal
-OR3-kEKt4IFkaqbvdHp	5/24/2025 10:36:35 PM	21.45	38.90	OK	False	False	False	False	Tray 1	22:36	22	Normal
-OR3-kFZCBiWolhA6Z6	5/24/2025 10:36:35 PM	21.45	38.94	OK	False	False	False	False	Tray 1	22:36	22	Normal
-OR3-kyu_LayhzhNAI	5/24/2025 10:36:37 PM	21.45	38.94	OK	False	False	False	False	Tray 1	22:36	22	Normal
-OR3-kk0JUAJ5s5zhc33	5/24/2025 10:36:37 PM	21.45	38.94	OK	False	False	False	False	Tray 1	22:36	22	Normal

Table: logs_output (28,358 rows)

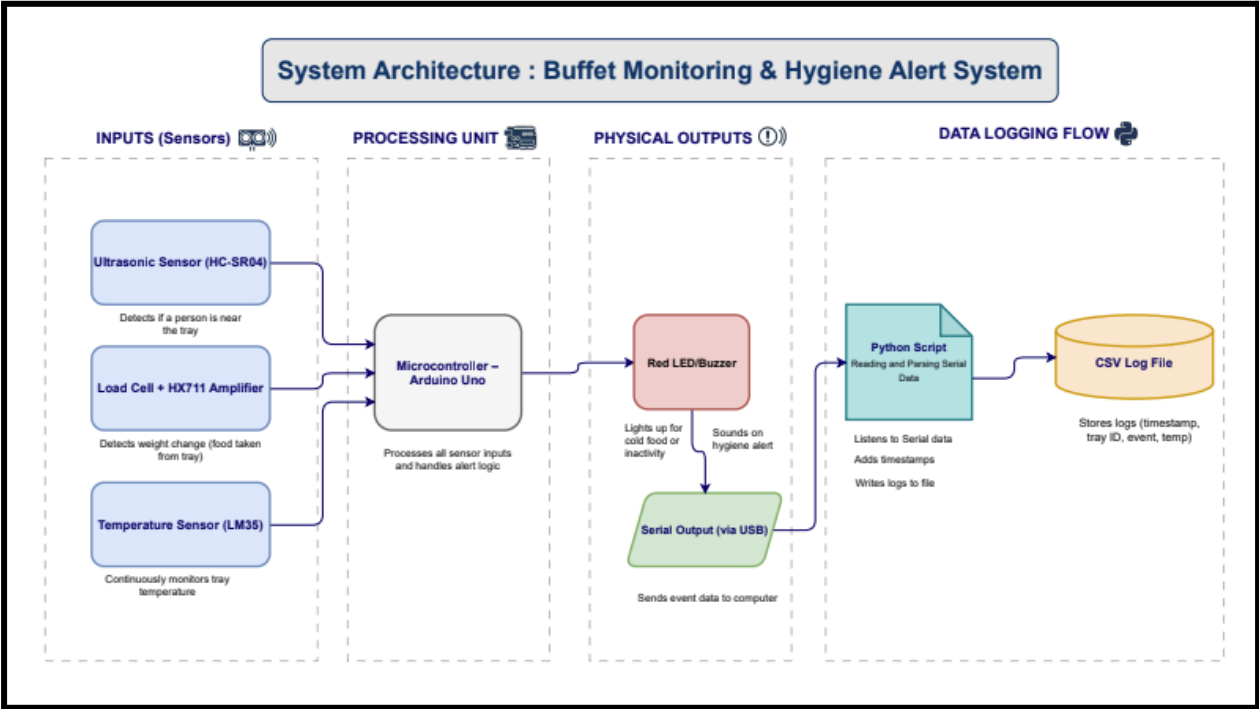
Data pane: logs_output, tray_history_output

tray_id	timestamp	status	confidence	override	TimeBucket
Tray 1	5/19/2025 7:45:47 AM	full	89.0	False	07:45
Tray 1	5/19/2025 7:47:09 AM	medium	77.7	False	07:47
Tray 1	5/19/2025 7:47:10 AM	full	63.7	False	07:47
Tray 1	5/19/2025 7:47:12 AM	full	66.5	False	07:47
Tray 1	5/19/2025 7:47:15 AM	full	63.0	False	07:47
Tray 1	5/19/2025 7:47:24 AM	medium	77.3	False	07:47
Tray 1	5/19/2025 7:47:25 AM	full	83.8	False	07:47
Tray 1	5/19/2025 7:47:27 AM	medium	77.9	False	07:47
Tray 1	5/19/2025 7:47:30 AM	full	82.9	False	07:47
Tray 1	5/19/2025 7:47:43 AM	medium	73.7	False	07:47
Tray 1	5/19/2025 7:47:44 AM	full	90.6	False	07:47
Tray 1	5/19/2025 7:47:46 AM	medium	79.9	False	07:47
Tray 1	5/19/2025 7:47:47 AM	medium	74.9	False	07:47
Tray 1	5/19/2025 7:47:49 AM	full	86.8	False	07:47
Tray 1	5/19/2025 7:47:50 AM	medium	75.1	False	07:47
Tray 1	5/19/2025 7:48:07 AM	empty	84.1	False	07:48
Tray 1	5/25/2025 12:07:42 AM	full	85.2	False	00:07
Tray 1	5/25/2025 1:23:17 PM	full	98.6	False	13:23
Tray 1	5/25/2025 2:25:44 PM	full	100.0	False	14:25
Tray 1	5/25/2025 2:26:12 PM	empty	83.4	False	14:26
Tray 1	5/25/2025 2:38:38 PM	full	100.0	False	14:38
Tray 1	5/25/2025 2:56:09 PM	full	99.9	False	14:56
Tray 1	5/25/2025 3:01:31 PM	full	96.9	False	15:01
Tray 1	5/25/2025 3:04:48 PM	empty	98.3	False	15:04
Tray 1	5/25/2025 3:06:02 PM	full	83.7	False	15:06
Tray 1	5/25/2025 3:14:29 PM	empty	74.7	False	15:14
Tray 1	5/25/2025 3:16:16 PM	full	92.5	False	15:16

Table: tray_history_output (27 rows)



Appendix I: Initial Smart Buffet Monitoring System Architecture Design



Appendix J: Final Smart Buffet Monitoring System Architecture Design

